



UNIVERSIDAD
DE GRANADA

Escuela Técnica Superior de Ingenierías Informática y de
Telecomunicación

MÁSTER UNIVERSITARIO EN CIENCIA DE DATOS E INGENIERÍA
DE COMPUTADORES

TRABAJO DE FIN DE MÁSTER

nois-learn: una biblioteca de python *scikit-learn*-like para el filtrado de ruido en datos tabulares

Análisis del uso de TabPFN en limpieza de ruido y su explicabilidad

Presentado por:
Juan Miguel Padilla Galera

Curso académico 2025-2026



nois-learn: una biblioteca de python
scikit-learn-like para el filtrado de ruido en
datos tabulares

Análisis del uso de TabPFN en limpieza de
ruido y su explicabilidad

Juan Miguel Padilla Galera

Juan Miguel Padilla Galera *nois-learn: una biblioteca de python scikit-learn-like para el filtrado de ruido en datos tabulares*
Análisis del uso de TabPFN en limpieza de ruido y su explicabilidad.

Trabajo de fin de Máster. Curso académico 2025-2026.

**Responsable de
tutorización**

Julián Luengo Martín
*Departamento de Ciencias de la
Computación e Inteligencia Artificial*

Máster Universitario en
Ciencia de Datos e
Ingeniería de
Computadores

Escuela Técnica Superior
de Ingenierías Informática
y de Telecomunicación
Universidad de Granada

Contents

1	Introducción	3
1.1	Motivación	3
1.2	Propuesta	5
1.3	Objetivos	5
1.4	Estructura a seguir	5
2	Estado del arte	7
2.1	Breve introducción al aprendizaje automático y el entrenamiento supervisado	7
2.2	Sobre la calidad de los datos	7
2.3	Ruido en el contexto de la clasificación supervisada	9
2.3.1	Ruido de atributos	10
2.3.2	Ruido de clase	11
2.4	Tratamiento de ruido	16
2.4.1	Aproximaciones para solventarlo	16
2.4.2	Sobre el filtrado como alternativa para el tratamiento de ruido	17
2.4.3	Taxonomía de los filtros de ruido	18
2.5	Análisis de las bibliotecas/frameworks destinados a la limpieza de datos ruidosos	19
3	Sobre TabPFN y nuestra propuesta como filtro de ruido	25
3.1	Sobre TabPFN	25
3.1.1	Sobre la aplicación del aprendizaje profundo en datos tabulares	25
3.1.2	PFNs	25
3.1.3	TabPFN	26
3.2	Propuesta: TabPFN sobre filtros basados en clasificadores	27
3.2.1	Motivación	27
3.2.2	Objetivos	27
4	Descripción de la biblioteca	29
4.1	Sobre la arquitectura y funcionamiento	29
4.2	Implementaciones de la biblioteca	29
4.2.1	Métodos de generación de ruido	30
4.2.2	Métodos de filtrado de ruido de clase	31
4.3	Ejemplo de uso	35
4.3.1	Uso básico de la biblioteca	35
4.3.2	Uso de la biblioteca con <i>scikit-learn</i>	36
4.4	Módulo de explicabilidad	37
4.4.1	Ejemplo de uso	40
4.5	Instalación de la biblioteca	42

5	Metodología de evaluación experimental	43
5.1	Evaluación de los filtros implementados en la biblioteca nois-learn	43
5.1.1	Conjuntos de datos	43
5.1.2	Técnicas de filtrado a evaluar	44
5.1.3	Metodología de evaluación	44
5.2	Evaluación de la propuesta	45
5.2.1	Conjuntos de datos utilizados	45
5.2.2	Metodología de simulación	45
5.2.3	Métricas de evaluación	46
5.2.4	Configuración de los filtros basados en distancias	46
5.2.5	Configuración de los filtros basados en clasificadores	47
5.2.6	Configuración de las técnicas de limpieza de clase	47
5.2.7	Metodología de evaluación de los resultados	47
5.2.8	Especificaciones del ordenador utilizado para realizar las simulaciones	50
6	Análisis de los resultados	51
6.1	Evaluación de la biblioteca	51
6.2	Evaluación de la propuesta	52
6.2.1	Desempeño global promedio	52
6.2.2	Desempeño frente a las demás técnicas	56
6.2.3	Análisis del tiempo de ejecución	60
6.2.4	Capacidad de detección de instancias ruidosas	60
7	Conclusiones	67
7.1	Conclusiones generales	67
7.2	Cumplimiento de objetivos	67
7.3	Limitaciones del trabajo	67
7.4	Líneas futuras	67
	Bibliography	69

1 Introducción

1.1 Motivación

Pese a la prolífica producción académica dentro del área de la minería de datos que, tal y como muestra la Figura 1.1, reúne más de 300 000 documentos en solo los últimos 3 años; así como pese al determinante papel que ha tomado el lenguaje de programación python a lo largo de la última década (tal y como muestra la Figura 1.2); hasta donde llega nuestro conocimiento (el de los autores), aún a día de hoy existen muy pocas bibliotecas que implementen en el lenguaje de programación antes mencionado una extensa ristra (siquiera alguna sola) de técnicas de detección y limpieza de datos tabulares ruidosos.

Ello no solo tiene un impacto negativo tanto en la producción científico-académica como en el sector empresarial en la medida en que aquel que necesite usarlas (las técnicas de filtrado de ruido) deberá, primero aprenderlas, posteriormente implementarlas, y finalmente confiar en tal implementación (lo que supone no solo un coste temporal, sino también cognitivo, pudiendo este ser más productivo en otras etapas de la experimentación y/o análisis); sino que también tiene un impacto en cuanto a estandarización de los procesos de investigación se refiere. Con esto último no nos referimos solo a la simplicidad de comprender la metodología seguida, sino también en lo que concierne a su reproducibilidad, algo que consideramos básico en un panorama investigador en el que la colaboración debería encontrarse a la orden del día; así como a su compatibilidad con otros estándares en el campo (como lo son bibliotecas como `numpy`, `pandas` o `scikit-learn`, las cuales se encuentran a la orden del día en lo que análisis de datos se refiere).

En un mundo en el que el almacenamiento digital se ha hecho cada vez más barato, compacto y eficiente, las empresas se han visto movidas por un síndrome de Diógenes digital que las ha instado a almacenar toda información que, quizás, pueda ser valiosa en un futuro (datos denominados por la empresa [Gartner](#) como *Dark Data*). El informe *Rethink*

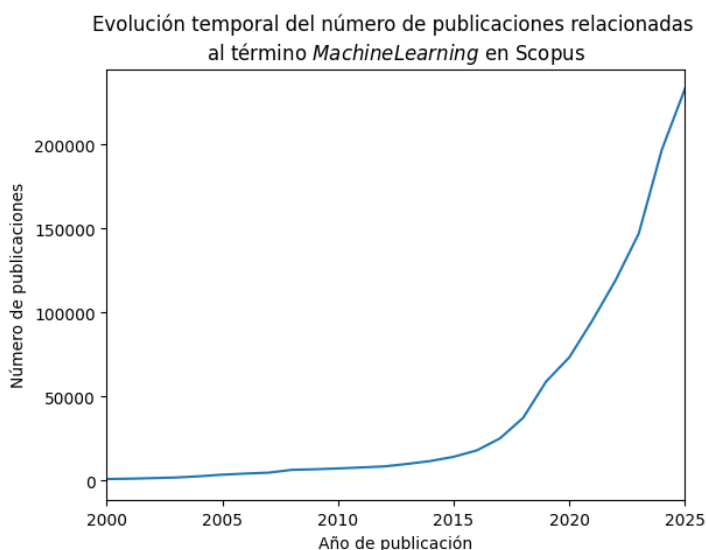


Figure 1.1: Evolución temporal de los documentos relacionados al término *Machine Learning* entre los años 2000 y 2025 en [Scopus](#).

1 Introducción

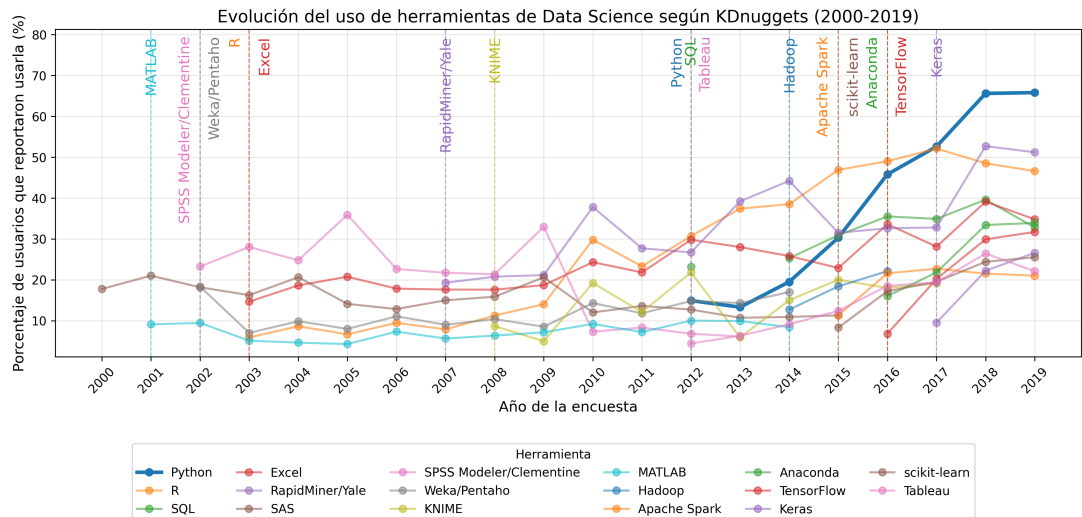


Figure 1.2: Evolución temporal del uso de las herramientas más usadas dentro de la minería de datos entre los años 2000 y 2019 según [kdnuggets](#).

Data[24], publicado en 2020 por la empresa de dispositivos de almacenamiento digital Seagate, reporta que solo el 32 % de los datos almacenados por empresas se pone realmente en uso. Estas, sin embargo, están comenzando a descubrir que no todo dato es valioso, que lo importante no es la cantidad de datos de que dispongamos, sino la calidad de los mismos, tal y como refleja recientemente mostraron Sambasivan et al. en [23], donde estudiaron los *Data Cascades*, definidos por ellos como "eventos acumulativos que provocan efectos negativos posteriores derivados de problemas en los datos" desencadenados por prácticas convencionales en IA/ML que misnuvaloran la calidad de los datos. En tales líneas los datos ruidosos se enmarcan dentro de aquellos por los que preocuparse a la hora de desarrollar tareas de aprendizaje automático. De manera introductoria, este tipo de datos se corresponde con aquellos que infieles a la realidad que tratan de representar. Ello en la medida en que, en conjunto, son incoherentes, por ejemplo, no siguiendo los patrones esperados al observar la mayoría de los demás datos disponibles.

En el mundo del aprendizaje automático, basado en gran medida en algoritmos que intentan identificar y replicar patrones en datos previamente recogidos, este fenómeno, tratado de manera indebida y, más específicamente, despreocupada o inconsciente, puede tener resultados considerablemente negativos, cuando no catastróficos. Y es que sin el debido preprocesamiento de los datos a emplear en la tarea objetivo, por la simple naturaleza de la mayoría de técnicas de aprendizaje automático, estas a priori confiarán en aquello con lo que las nutramos. Es decir, dichas técnicas tratarán de adaptarse a los datos con las que las alimentemos sin, en la mayoría de los casos, cuestionarse la veracidad de los mismos.

Se hace así cada vez más patente que, al igual que el reescalado de las variables numéricas o el tratamiento de valores perdidos componen un estándar básico en cualquier etapa de extracción de conocimiento a partir de datos, el tratamiento de ruido (así como cuando corresponde también, el del desequilibrio entre clases o el de *fairness*) no es menos.

1.2 Propuesta

Con ello este trabajo se propone un problema que creemos de gran interés académico e investigador, en tanto en cuanto acelera y optimiza aquellas partes usualmente menos productivas y más repetitivas de la tarea. Más específicamente se plantea primeramente abordar el problema anterior mediante el diseño una biblioteca que no solo ahune una extensa ristra de técnicas útiles y reconocidas para la detección, filtrado y limpieza de datos ruidosos; sino que además lo haga siguiendo la arquitectura y los estándares de la archiconocida librería [scikit-learn](#), diseñando con ello funciones y objetos compatibles con la misma, siguiendo una funcionalidad y sintaxis prácticamente idénticas.

Junto a lo anterior, estudiaremos el potencial del clasificador TabPFN [11] (por Tabular Prior-Fitted-Networks, técnica estado del arte en el contexto del aprendizaje automático sobre datos tabulares) en el etiquetado de instancias potencialmente ruidosas, algo que hasta donde alcanza nuestro conocimiento no ha sido realizado con anterioridad. No solo realizaremos una comparación del mismo frente a los filtros de ruido más ampliamente usados, sino que además se explorará la explicabilidad del mismo (es decir, de las decisiones que toma a la hora de etiquetar o no una instancia como ruidosa) a través de los coeficientes SHAP [17] (por SHapley Additive exPlanations).

1.3 Objetivos

Con este trabajo de fin de máster nos proponemos así abordar los siguientes objetivos:

- Diseñar una biblioteca de python *scikit-learn-like* que cuente con los filtros de ruido clásicos más conocidos (basados en distancias, como ENN, Multiedit, NCNEdit ...; así como en clasificadores, como CF, EF, CVCF, IPF, ...) junto con algunas propuestas más actuales como CNC-NOS [16],
- Validar el correcto y esperado desempeño de los filtros anteriores sobre una variada (por su número de instancias, atributos numéricos, atributos categóricos, dimensiones totales) ristra de conjuntos de datos.
- Implementar y analizar el desempeño de filtros de ruido basados en el clasificador TabPFN, así como su comparación frente a una selección de los anteriores. Adicionalmente nos proponemos añadir y explotar un módulo de explicabilidad que fundamente las decisiones tomadas por el filtro, ello con el objetivo indirecto de ayudar en la labor del analista reduciendo el conocimiento experto necesario.

Añadir más
métodos SOTA

1.4 Estructura a seguir

Comenzaremos trabajo en la Sección 2 introduciendo el estado del arte del filtrado de conjuntos de datos ruidosos, definiendo formalmente el concepto de ruido así como los

Repsar esto,
pues depende
de como se haga
el trabajo final

tipos existentes en la literatura y las diferentes aproximaciones para solventar el mismo.

Posteriormente en la Sección 4 se presentará la aquí propuesta biblioteca de python `nois-learn`. Más específicamente, se discutirá su arquitectura, en comparación con la de la librería `scikit-learn`, así como su uso y las diferentes técnicas de filtrado y generación de ruido implementadas hasta la fecha de entrega de este artículo. Finalizando la sección con un ejemplo de uso de la misma biblioteca.

A continuación, en la Sección 3 propondremos una novedosa técnica de filtrado de conjuntos de datos basada en el empleo de Tab-PFN[11], hoy día establecido como algoritmo estado del arte en las tareas de predicción en conjuntos de datos tabulares.

Mostrado lo anterior, en la Sección 5 pasaremos a presentar la metodología a seguir de cara a la evaluación del correcto diseño y funcionamiento de las técnicas de filtrado implementadas en la librería propuesta, así como de la técnica antes propuesta.

Aclarada la metodología experimental a seguir, en la sección 6 se continuará con la presentación y discusión de los resultados obtenidos. Concretamente se hará un análisis de desempeño promedio de una selección de las técnicas de filtrado implementadas. Este análisis será realizado tanto a nivel de familia (estudiando el desempeño en conjunto de filtros basados en una misma técnica, pero con diferentes hiperparametrizaciones) como a nivel de hiperparametrización (es decir, diferenciando filtros no solo por el tipo, sino además por la hiperparametrización subyacente). Realizaremos con ello un análisis del impacto en la etapa de predicción posterior al filtrado de los datos (examinando las mejoras promedio obtenidas en el accuracy balancead), además de en el etiquetado de instancias ruidosas (a partir de del par precision-recall¹ junto con el MCC —Matthews Correlation Coefficient—).

Comprobar si se añaden más cosas

Completar cuando se haya acabado

Por último finalizaremos en la Sección , reflexionando sobre todo lo desarrollado en el trabajo ...

¹Referente al etiquetado de instancias ruidosas. Es decir, analizaremos el precision y recall de los filtros a la hora de captar instancias verdaderamente ruidosas, sin tener nada que ver este par de métricas con la etapa de clasificación posterior al preprocesamiento de los datos a filtrar.

2 Estado del arte

2.1 Breve introducción al aprendizaje automático y el entrenamiento supervisado

El aprendizaje automático, o *Machine Learning*, constituye una rama de la inteligencia artificial centrada en el desarrollo de métodos capaces de extraer patrones a partir de datos y emplearlos para realizar predicciones, tomar decisiones o descubrir estructuras subyacentes en la información disponible. A diferencia de los enfoques clásicos basados en reglas explícita y previamente programadas, los algoritmos de aprendizaje automático construyen modelos a partir de conjuntos de ejemplos ajustando sus parámetros de acuerdo con la evidencia contenida en los datos previamente introducidos [19, 3].

De forma general, los problemas y/o técnicas de aprendizaje automático suelen clasificarse en tres grandes paradigmas: aprendizaje supervisado, aprendizaje no supervisado y aprendizaje por refuerzo:

- En el aprendizaje no supervisado, el objetivo consiste en descubrir patrones, agrupaciones o representaciones latentes sin disponer de una variable objetivo conocida a priori, siendo a veces así considerada como una fuente de extracción de conocimiento.
- Por otro lado el aprendizaje por refuerzo consta de un agente que incrementalmente aprende a tomar decisiones mediante la constante y sucesiva interacción con un entorno, recibiendo recompensas o penalizaciones en función de sus acciones, a partir de las que adaptar su comportamiento de cara a futuras ejecuciones.
- Por último, el aprendizaje supervisado engloba aquellos algoritmos que, a partir de un conjunto de entrenamiento ($TR = (x_i, y_i)_{i=1}^n$), formado por instancias (x_i) y sus correspondientes salidas esperadas (y_i), buscan aprender una función ($\hat{f}$) capaz de generalizar dicha relación a nuevos datos no observados previamente [12]. De este modo, el objetivo consiste en obtener un modelo que, dada una nueva instancia (x_i), produzca una predicción ($\hat{y}_i = \hat{f}(x_i)$) lo más próxima posible a la salida real esperada (y_i). Esta proximidad se mide mediante una función de pérdida o error, cuya forma concreta depende del tipo de problema considerado, siendo habitual distinguir entre tareas de regresión, cuando la variable objetivo es numérica, y tareas de clasificación, cuando la salida pertenece a un conjunto discreto de clases.

2.2 Sobre la calidad de los datos

En el contexto del aprendizaje supervisado, en el que como se ha expuesto antes los modelos aprenden a tomar decisiones en base a los datos con que son entrenados, la calidad de los datos con que se alimenten los mismos toma consecuentemente un papel muy relevante,

teniendo estos una influencia directa sobre el comportamiento de los modelos.

En tal sentido, un conjunto de datos puede ser de baja calidad desde diversas perspectivas, como:

- **Compleitud:** haciendo referencia a la presencia de toda la información necesaria para describir correctamente todas y cada una de las instancias a emplear en la tarea objetivo. En tal sentido encontramos problemas como la existencia de valores ausentes y registros incompletos, que puede reducir la cantidad efectiva de información disponible, dificultando el entrenamiento de determinados algoritmos [13].
- **Consistencia y validez:** en referencia a que los datos respeten una estructura homogénea y coherente. Problemas como formatos heterogéneos para un mismo atributo y codificaciones inconsistentes (como el registro de fechas usando de manera simultánea los formatos DD/MM/YY y MM-DD-YY), unidades mezcladas o valores fuera del dominio esperado pueden dificultar y hasta imposibilitar del proceso de aprendizaje [29, 26].
- **Unicidad y redundancia:** un conjunto de datos puede contener registros duplicados o atributos que, en función del modelo sobre el que se apliquen, pueden aportar información redundante. Estas situaciones pueden distorsionar la distribución empírica del problema, asignando un peso excesivo a determinadas configuraciones, desmbocando en modelos sesgados.
- **Ruido:** los datos pueden contener alteraciones e incoherencias que dificultan la identificación de patrones fiables. Este ruido puede afectar tanto a los atributos de entrada como a las etiquetas asociadas a las instancias. Ello puede llevar a los algoritmos supervisado a inducir fronteras de decisión o patrones entre atributos erróneos, reduciendo con ello la capacidad de generalización del modelo [32, 8].
- **Relevancia y adecuación:** incluso si un conjunto de datos es formalmente correcto, sin ningún tipo de falta en como ha sido registrados, puede no ser útil para una tarea concreta si las variables disponibles no guardan relación suficiente con el objetivo predictivo y no responden a las necesidades del problema considerado. Por ello, la calidad de los datos debe valorarse también en función de su adecuación al uso previsto [30, 26].
- **Representatividad:** de manera similar al punto anterior, no basta con seleccionar aquellos atributos que más se relacionen con la tarea objetivo, sino que se requiere de un conjunto de entrenamiento que cubra de forma suficiente las regiones relevantes del espacio de entrada. Una muestra poco representativa puede conducir a modelos con buen rendimiento aparente en promedio, pero con un pésimo desempeño cuando se despliega fuera del laboratorio [18].
- **Desequilibrio entre clases:** relacionado con el punto anterior, uno de los clásicos problemas especialmente en el contexto de algoritmos de supervisados de clasificación se encuentra en conjuntos de datos con un notable desequilibrio respecto de las clases a predecir (es decir, cuando el número de valores distintos que puede tomar la variable objetivo y_i es considerablemente diferente). Esta situación puede provocar que los clasificadores favorezcan la clase mayoritaria y obtengan un rendimiento pobre precisamente sobre las clases minoritarias, que en muchos dominios son las de mayor interés [10].

- **Sesgos y equidad:** de manera similar al caso anterior, puede existir un desequilibrio, o incluso un sesgo, no dentro de la variable a predecir, sino dentro de los atributos a partir de los que el modelo realiza sus predicciones. En tales líneas los datos pueden reflejar sesgos históricos, sociales o derivados del propio proceso de recolección, dando lugar a modelos que, incoherentemente, producen resultados sistemáticamente desfavorables para ciertos grupos. En un marco como el anterior la calidad de los datos deja de ser valiosa de cara solo al rendimiento predictivo, para tomar una gran importancia social a través de la equidad, robustez y fiabilidad del sistema aprendido [18, 27].

De modo que todo proceso de análisis y empleo de los datos en técnicas de aprendizaje automático que aspire a obtener resultados, no solo potencialmente buenos, sino, además, justos, exige un previo y cuidadoso estudio sobre los factores anteriores.

Revisar detenidamente cada una de las refs aquí mencionadas, aunque a priori parecen cumplir con lo previsto

2.3 Ruido en el contexto de la clasificación supervisada

Retomando el punto referente al ruido (en datos) de la sección anterior, de manera breve podríamos enmarcar el mismo como errores, inconsistencias o alteraciones en conjuntos de datos, los cuales dificultan la identificación de patrones fiables.

Formalmente, sea un conjunto de datos supervisado formado por n instancias, donde cada instancia limpia o ideal viene dada por el par

$$(x_i, y_i) \in \mathcal{X} \times \mathcal{Y},$$

siendo

$$x_i = (x_{i1}, x_{i2}, \dots, x_{im})$$

el vector de m atributos descriptivos, e y_i la etiqueta o variable objetivo asociada. En un escenario ideal, los pares observados en el conjunto de entrenamiento coincidirían con dichos pares limpios. Sin embargo, en presencia de ruido, el conjunto de datos disponible no contiene necesariamente los pares (x_i, y_i) , sino una versión observada y potencialmente alterada,

$$(\tilde{x}_i, \tilde{y}_i),$$

obtenida mediante algún mecanismo de corrupción o perturbación.

Desde esta perspectiva, puede decirse que una instancia presenta ruido cuando existe una discrepancia entre la información limpia que idealmente debería representar y la información finalmente observada en el conjunto de datos, es decir, mientras se cumple que

$$(\tilde{x}_i, \tilde{y}_i) \neq (x_i, y_i).$$

Dicha discrepancia puede afectar tanto a los atributos de entrada ($\tilde{x}_{ij}$) como al de salida ($\tilde{y}_i$), incluso simultáneamente. Así, se habla de ruido en atributos cuando

$$\tilde{x}_i \neq x_i,$$

es decir, mientras dos registros que deberían ser idénticos difieren en una o más de las m dimensiones que los definen.

Mientras que se habla de ruido de clase o ruido de etiqueta cuando

$$\tilde{y}_i \neq y_i.$$

Por tanto, en el contexto del aprendizaje supervisado, el ruido de datos puede entenderse como la existencia de perturbaciones en los atributos, en las etiquetas o en ambos, que provocan que el conjunto de datos observado no refleje fielmente la distribución subyacente que se desea aprender. Esta alteración dificulta la identificación de patrones fiables y puede afectar negativamente al proceso de entrenamiento y a la capacidad de generalización de los modelos inducidos [32, 8].

Abordemos a continuación más detalladamente cada uno de los tipos de ruido antes expuesto.

2.3.1 Ruido de atributos

Como se ha expuesto antes, el ruido de atributos (conocido en la literatura anglosajona por *attribute noise* o *feature noise*), aparece cuando uno o varios valores de los atributos descriptivos x_{ij} de una instancia x_i son incorrectos, imprecisos, incompletos o han sido alterados de alguna forma. En este caso, la etiqueta de clase asociada y_i a dicha instancia puede ser correcta, pero la información utilizada para describir la instancia (sea esta la lista $\{x_{ij}\}_{j=1}^{j=m}$) contendrá errores en aquellos atributos de entrada ruidosos.

Por la naturaleza de los modelos de clasificación supervisada, cuyos algoritmo tratan de identificar patrones, regularidades y relaciones útiles en los atributos de entrada para separar unas clases de otras, la presencia de ruido en estos (los atributos de entrada) puede llevar a los modelos a percibir una representación distorsionada de los ejemplos que conforman el conjunto de entrenamiento ($TR = \{x_i\}_{i=1}^{i=n}$), aprendiendo con ello relaciones espúreas o menos precisas entre las variables de entrada y la de salida.

Este tipo de ruido puede tener distintos orígenes, que van desde problemas de registro asociados a errores en la de medición (por sensores defectuosos, por ejemplo) o valores introducidos manualmente de forma incorrecta (voluntaria o involuntariamente¹), hasta problemas en el redondeo de los datos, codificaciones inconsistentes o integración de fuentes heterogéneas, entre otros motivos.

Tanto atributos numéricos como categóricos pueden verse afectados por este fenómeno. En atributos numéricos puede aparecer como pequeñas perturbaciones respecto del valor real, valores extremos o errores de escala; mientras suele manifestarse mediante categorías mal codificadas o etiquetadas.

¹Un caso habitual es el sesgo de deseabilidad social, por el cual los encuestados tienden a proporcionar respuestas que proyectan una imagen más favorable de sí mismos[2]. Esto puede provocar que determinados comportamientos socialmente sensibles, como fumar, consumir alcohol o realizar ciertas prácticas de riesgo, sean infradeclarados. En consecuencia, el conjunto de datos puede contener valores de atributos incorrectos, aunque hayan sido introducidos aparentemente de forma válida.

En cualquier caso debe siempre procurarse no confundirlos con las instancias atípicas (u *outliers* en la lengua inglesa), las cuales, pese presentar uno o más atributos con un valor totalmente alejado de la tendencia global, no por ello dejan de ser muestras total y completamente válidas, pudiendo estas en realidad contener información no contemplada en un inicio. En relación a lo anterior sucede muchas veces que el problema no se encuentra tanto en la misma instancia calificada como atípica, sino en el conjunto de atributos que se usa para definirla, exhibiendo con ello una falla en el previo registro de los mismos.

Desde el punto de vista del aprendizaje, el ruido de atributos distorsiona el espacio de características sobre el que trabaja el modelo. Tal y como muestra la Figura 2.1 (donde se refleja el impacto en la distribución de los datos así como en la frontera de decisión modelada por una SVM antes y después de haber sido entrenada con instancias ruidosas), una instancia afectada por ruido de atributo puede desplazarse artificialmente hacia una región del espacio correspondiente a otra clase o comportarse como un caso atípico totalmente alejado de los demás (tal y como sucede con la instancia marcada en lila en la misma Figura). Como se muestra en dicho gráfico, esto puede desembocar en fronteras de decisión menos estables, pérdida de capacidad discriminativa y reducción del rendimiento predictivo.

Así, aunque el ruido de atributos no modifique directamente la clase objetivo, puede perjudicar de forma significativa el proceso de su aprendizaje, especialmente cuando afecta a variables muy relevantes para la clasificación. Su impacto depende con ello no solo de la importancia de la variable afectada, sino también del número de atributos alterados y del nivel en que han sido alterados.

No obstante, tal y como se ha expuesto antes, es importante actuar con suma cautela a la hora de aplicar de técnicas enfocadas en solventar este problema, pues no siempre una instancia con valores poco frecuentes será necesariamente ruidosa. Sino que podrá representar un caso minoritario, una región poco poblada (o poco densa) del espacio de entrada o una situación real, pero excepcional, que, en cualquiera de los casos antes propuestos, el modelo debe aprender.

El tratamiento del ruido de atributos supone así una constante búsqueda por equilibrar la eliminación y/o corrección de información defectuosa con la conservación de ejemplos válidos que aporten diversidad al conjunto de entrenamiento.

2.3.2 Ruido de clase

El ruido de clase aparece cuando una instancia tiene asignada una etiqueta (de salida, a predecir por la técnica de aprendizaje supervisado correspondiente) incorrecta.

En clasificación supervisada, la clase y_i asociada a cada ejemplo x_i actúa como la respuesta correcta durante el entrenamiento. El algoritmo utiliza esa información para ajustar sus parámetros, construir reglas de decisión o definir fronteras entre clases. Por tanto, cuando una etiqueta es errónea, el modelo recibe una señal de aprendizaje incorrecta. Así en este caso la instancia puede estar correctamente descrita por un atributos de entrada x_{ij} completamente coherentes entre sí, pero asociados estos a una clase que no se corresponde con su

Efecto del ruido en la frontera de decisión de una SVM

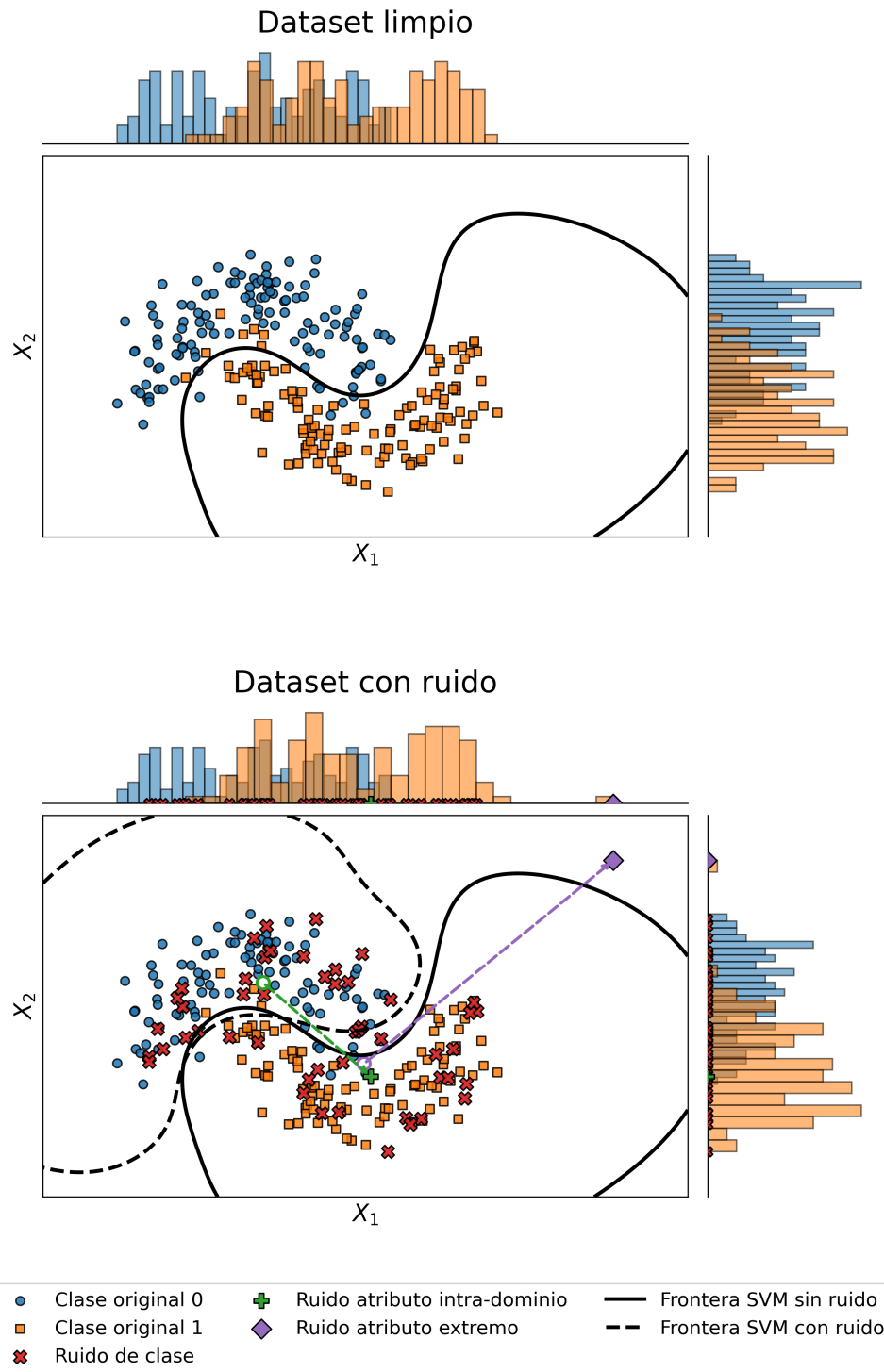


Figure 2.1: Impacto en la distribución de los datos así como en la frontera de decisión predicha por una SVM al introducir ruido al conjunto de entrenamiento. Al contar este con 2 clases de salida, el ruido intra-dominio ha sido introducido intercambiando de modo aleatorio un 20% de las etiquetas de salida por su valor alternativo; mientras que el ruido extremo se ha hecho desplazando los atributos de una de las instancias lejos del dominio definido por el conjunto original

verdadera naturaleza (es decir, diferente de la que realmente le corresponde).

De modo similar (cuando no igual) al ruido de atributo, el de clase puede producirse por errores humanos de anotación, ambigüedad entre categorías, discrepancias entre expertos, cambios en los criterios de clasificación, entre otros motivos. Por ejemplo, en un problema médico un paciente (x_i) puede tener correctamente registrados sus atributos clínicos (x_{ij}) pero estar asociado a un diagnóstico equivocado (y_i). En ese caso, el problema no estaría en la descripción de la instancia, sino en la etiqueta utilizada como salida esperada.

Este tipo de ruido suele considerarse especialmente perjudicial, pues mientras que el de atributo puede verse parcialmente relajado en problemas en los que se manejen varias variables de entrada o en los que el atributo ruidoso no es de gran importancia, en este caso la imprecisión en la anotación de la clase asociada a una instancia altera directamente el objetivo del aprendizaje, pues sesga las recompensas que el modelo recibe en el mismo. Como consecuencia, el clasificador puede por ejemplo interpretar como válidas relaciones que en realidad son erróneas así como ajustar incorrectamente fronteras de decisión.

Análogamente, el impacto del ruido de clase depende del porcentaje de etiquetas erróneamente registradas así como del nivel de error (en tanto en cuanto si una instancia ruidosa se encuentra rodeada de muestras pertenecientes a otra clase, en una frontera, ...). Una etiqueta incorrecta en una zona densamente poblada puede generar contradicciones evidentes con las instancias vecinas. En cambio, una etiqueta incorrecta cerca de una frontera de decisión puede ser más difícil de detectar, ya que puede confundirse con un ejemplo difícil o ambiguo.

En la literatura suele realizarse una distinción entre ruidos de clase tal y como sigue:

- **Ruido completamente aleatorio:** aparece cuando cualquier etiqueta puede ser alterada con la misma probabilidad, independientemente de la clase original a la que perteneciera así como de los atributos asociados a la instancia.
- **Ruido dependiente de la clase:** se produce cuando algunas clases (del dominio que puede tomar y_i) tienen mayor probabilidad de ser alteradas que otras².
- **Ruido dependiente de los atributos:** aparece cuando la probabilidad de que una etiqueta y_i sea alterada depende de las características de entrada (x_{ij}) concretas de la instancia. Esto puede suceder por ejemplo en aquellos diagnósticos hechos a sobre paciente con atributos insuficientemente determinantes, situando al paciente cerca de las fronteras de decisión, dificultando así la labor del experto, quien puede errar en su diagnóstico final.

Existe también la división del ruido entre completamente aleatorio (o *NCAR* en inglés, por *Noise Completely At Random*), dependiente de la clase (o *NAR* en inglés, por *Noise At Random*) o dependiente de la clase y la combinación de atributos (o *NNAR* en inglés, por *Noise Not At Random*).

La presencia de ruido de clase justifica así el uso de técnicas de tratamiento de ruido, especialmente métodos de filtrado de instancias, los cuales intentan identificar ejemplos cuya

²Como puede suceder voluntariamente como causa del sesgo de deseabilidad antes mencionado.

etiqueta parece inconsistente respecto al comportamiento del resto del conjunto de datos. Para ello estas técnicas suelen comparar la clase asignada a una instancia con la clase estimada por sus vecinos haciendo uso de clasificador auxiliar o por un conjunto de los mismos.

Sin embargo, nuevamente, eliminar instancias sospechosas también implica cierto riesgo. No toda instancia mal clasificada por un filtro es necesariamente ruidosa. Algunas instancias pueden ser ejemplos raros, minoritarios o difíciles, pero válidos, como lo pueden ser clases minoritarias o situadas en regiones de menor densidad que las clases que las rodean.

Sobre la diferenciación entre ruido de clase y de atributo

De manera muy breve queremos abordar aquí un concepto que creemos relevante en referencia a la doble clasificación del ruido presentada anteriormente.

Y es que consideramos que, en la práctica, dentro del contexto de la clasificación supervisada, el ruido de atributo (al menos aquel con un impacto negativo en la fase de entrenamiento de modelos predictivos) es en general indistinguible de el de clase, salvo en aquellos casos en los que el ruido de atributo está asociado a valores extremos, muy alejados del dominio definido por la mayor parte del conjunto. Es decir, el ruido de atributo que modifica los valores x_{ij} que definen a una instancia x_i , desplazándola en el espacio vectorial $X = X_1 \times X_2 \times \dots \times X_n$ compuesto por las dimensiones asociadas a cada uno de los atributos $x_{ij} \in X_j$ desde una región poblada mayoritariamente por una clase c_i hacia otra región hacinada de instancias de otra clase diferente $c_k : i \neq k$, sería en cierto modo equivalente a intercambiar la clase c_k de la instancia más vecina a la anteriormente desplazada por la clase c_i . En ambos casos se tendría anómalamente una instancia perteneciente a la clase c_i dentro del dominio definido por la clase $c_k : k \neq i$ ³.

Estamos con ello comparando un intercambio en los atributos de una instancia (ruido de atributo) c_i , con una alteración de la clase de otra instancia diferente c_k , a la que asignamos la clase c_i .

Como con el bloque de abajo, podría acompañarse esto de una gráfica

Bien es cierto que puede seguir existiendo ruido de atributo sin estar este asociado a registros con valores extremos o desplazados a dominios poblados por otro tipo de clases (es decir, instancias con algún atributo incorrectamente registrado, pero contenidas en el dominio de la clase a que pertenecen). Sin embargo, por continuar situándose dentro de un rango coherente de valores (considerando todos los atributos de entrada junto con la etiqueta de salida), este tipo de ruido, si siquiera fuera detectable, en principio no debería tener un gran (si acaso algún) impacto negativo en las etapas de entrenamiento y validación, al, pese haber sido registrado erróneamente, seguir reflejando una instancia viable y coherente.

Esto último se ejemplifica en la figura 2.2, en la que, en orden descendente, se grafica primero una distribución de puntos arbitraria inicial, a continuación la misma con pequeñas variaciones en los atributos (lo que equivale al desplazamiento de los puntos en el espacio bidimensional en que está contenidos) de un 20% de las instancias (seleccionadas aleatoria e independientemente de su etiqueta de salida asociada), para acabar mostrando en la última

³Salvando el hecho de que existe una diferencia entre ambas situaciones en el grado en que en la primera se ve reducida en una unidad el número de instancias de la clase c_i ; mientras que en el segundo caso se ve reducida en la misma cantidad el número de instancias de la clase c_j

Ruido de atributo intra-clase en un conjunto de datos bidimensional

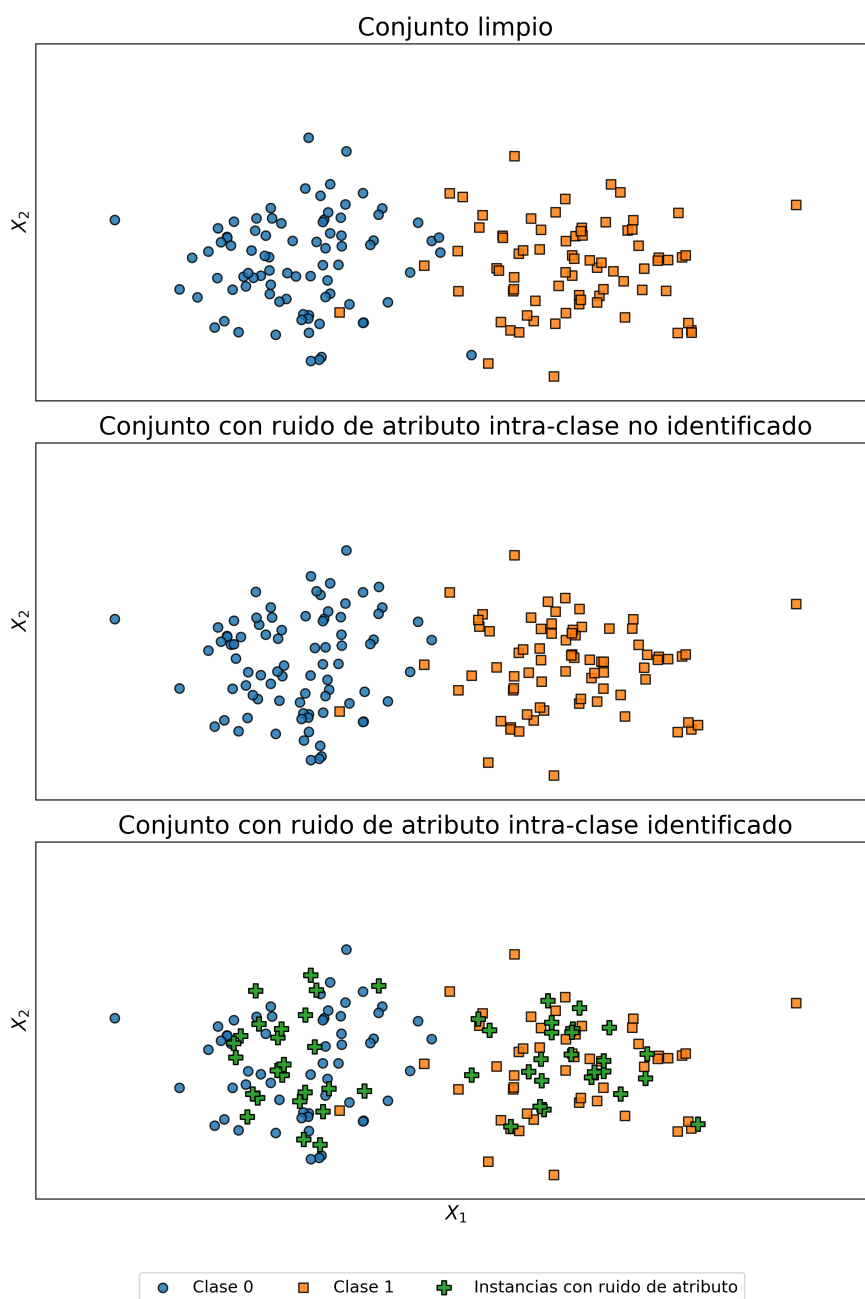


Figure 2.2: Representación del cambio en una distribución de puntos al introducir ruido de atributo que sin embargo mantiene cada instancia aproximadamente dentro del dominio original ocupado por su clase asociada.

gráfica la lo mismo que en la anterior, pero esta vez remarcando con cruces verdes aquellas instancias que han sido alteradas/desplazadas (a nivel de atributo). Si nos fijamos solo en las dos primeras gráficas notaremos la considerable dificultad existente de cara a atisbar cualquier diferencia entre ambas. Ambas distribuciones de puntos, pese a ser diferentes, globalmente son muy similares. No obstante no hay que dejar de lado que, pese a similares, ambas darán resultados diferentes a la hora de computar estadísticos (como la media o mediana), así como dará lugar a fronteras de decisión marginalmente diferentes. Pero tales diferencias no tendrán otro origen que la misma arbitrariedad existente a la hora de medir, que usualmente no da siempre los mismo resultados, pese a realizar mediciones en las mismas condiciones. Tanto es así que, una fuente de datos como la anteriormente propuesta se puede lograr en tales condiciones. Es decir, en ambos casos ambas distribuciones, siendo localmente diferentes y muy similares globalmente, representarán prácticamente la misma realidad.

Con esto no tratamos de renovar y/o derrocar la literatura que realiza la anterior división, pues dicha categorización es especialmente útil dentro del laboratorio, al realizar experimentos replicables a partir de datos hasta cierto punto sintéticos. Sino que tratamos de poner de manifiesto el problema epistemológico que existe al abordar conjuntos de datos reales con tal clasificación en mente.

Quizás pueda reformularse esto y añadir algo más. Conviene repasarlo con mayor detenimiento

2.4 Tratamiento de ruido

2.4.1 Aproximaciones para solventarlo

Como se ha expuesto repetidamente, la presencia de ruido en los datos puede afectar negativamente al proceso de aprendizaje (así como incluso a cualquier etapa de extracción de conocimiento basada en un análisis exploratorio de los datos — o *EDA*, en inglés —), reduciendo la capacidad predictiva de los modelos al dificultar la identificación de patrones fiables.

Debido a esta pérdida de rendimiento, diversos trabajos han señalado la necesidad de emplear técnicas específicas para mitigar los efectos del ruido en los datos [32]. En la literatura especializada se suelen distinguir dos grandes estrategias para abordar este problema:

- La adaptación de los algoritmos de aprendizaje para que sean más robustos frente al ruido (como lo son los *RandomForest*).
- El preprocesamiento de los datos con el objetivo de detectar, corregir o eliminar las instancias problemáticas.

La primera estrategia consiste en modificar o diseñar algoritmos capaces de manejar adecuadamente la presencia de ruido durante el propio proceso de aprendizaje. Estos métodos suelen denominarse *robust learners*, ya que tratan de reducir la influencia de los ejemplos ruidosos sobre el modelo final. Algunos algoritmos clásicos, como C4.5 [22] o determinados métodos de inducción de reglas [7], incorporan mecanismos que pueden hacerlos menos sensibles a ciertos tipos de ruido. Sin embargo, esta aproximación presenta una limitación importante: su efectividad depende directamente del algoritmo utilizado, no pudiendo

necesariamente trasladar la misma solución a problemas en los que se requieren otros modelos de aprendizaje (donde deberíamos considerar la dificultad o siquiera posibilidad inherente de adaptar un algoritmo existente).

La segunda estrategia se basa en actuar sobre los datos disponibles de cara al entrenamiento del modelo. En este caso, se introduce una etapa de preprocesamiento cuyo objetivo es identificar las instancias sospechosas de contener ruido, para posteriormente eliminarlas o corregirlas [4, 9]. Esta aproximación tiene la ventaja de ser agnóstica al clasificador que le suceda, ya que el aprendizaje se realiza sobre un conjunto de datos previamente depurado. No obstante este acercamiento presenta también inconvenientes, pues puede incrementar el coste computacional del proceso al añadir una fase adicional previa al entrenamiento, así como que muchos de estos métodos están diseñados para detectar un tipo concreto de ruido, no siendo con ello globalmente generalizables, lo que puede dificultar el problema cuando se desconoce la naturaleza del ruido subyacente en los datos a emplear.

Resulta así especialmente interesante estudiar mecanismos que permitan reducir sus efectos sin necesidad de modificar cada algoritmo de clasificación de forma individual y sin asumir de antemano un tipo o nivel concreto de ruido en los datos. En este contexto, los métodos de filtrado aparecen como una alternativa especialmente adecuada, ya que separan la fase de detección de ruido de la fase de aprendizaje del modelo (lo que adicionalmente nos da la oportunidad de estudiar individualmente el filtrado realizado, pudiendo ser este útil también de cara a otro tipo de aplicaciones fuera del dominio del entrenamiento de modelos, como para su uso en análisis exploratorios de datos —EDA—, de acuerdo a lo expuesto antes). En las siguientes secciones profundizaremos más en la segunda de las alternativas.

2.4.2 Sobre el filtrado como alternativa para el tratamiento de ruido

El objetivo de los algoritmos de filtrado de ruido es siempre el mismo: introducido un conjunto de datos crudo, marcar como potencialmente ruidosas, aquellas instancias que bajo el criterio del algoritmo lo sean. De modo que tras el filtrado se dispondría de los identificadores de las instancias limpias y de las ruidosas (o cualquier resultado equivalente, como lo puede ser tener el mismo conjunto de datos limpio y ruidoso, en lugar de los identificadores de los registros a partir de los que cada uno se compone). Lo que suele así diferenciar una técnica de otra es el criterio en el que basa sus decisiones. Pero antes de indagar en las distintas estrategias comunmente usadas, presentemos formalmente algunos conceptos relacionados con el tema.

Formalmente, en la descripción de los métodos de filtrado de ruido suele denotarse por $TR = \{x_i\}_{i=1}^{i=n}$ (por *train set*) al conjunto de entrenamiento original (al crudo, sin ninguna limpieza de ruido previamente realizada), por FS (por *filtered set*) al conjunto filtrado resultante (de aplicar la técnica de filtrado de ruido seleccionada) y por ND (por *noisy detected*) al conjunto de instancias identificadas como potencialmente ruidosas (por el filtro empleado). De modo que, tras aplicar el filtro se cumple que $TR = FS \cup ND$ y $\emptyset = FS \cap ND$.

Se tiene así que, previamente a aplicar el filtro, el conjunto ND se encuentra vacío ($ND = \emptyset$), ya que ninguna instancia ha sido todavía marcada como ruido. En contraposición, el conjunto FS se corresponde inicialmente con el conjunto de entrenamiento completo (es de-

cir, previamente a aplicar el filtro $FS = TR$). De modo que durante la ejecución del filtro este procesará (en una o más iteraciones, en función de la técnica aplicada) las instancias remanentes en el conjunto FS , etiquetando como potencialmente sospechosas parte las mismas, las cuales serán en tal caso removidas del conjunto FS y trasladadas al conjunto ND . No obstante existen métodos que realizan el etiquetado en varias fases, siguiendo más de un criterio, en cuyo caso consideraríamos solo la decisión final de cara a la modificación de los conjuntos FS y ND ⁴.

Una vez computados los conjuntos de entrenamiento limpio FS y ruidoso ND , los algoritmos de filtrado tienen dos alternativas:

- Eliminar las instancias marcadas como potencialmente ruidosas.
- Reetiquetar las instancias marcadas como potencialmente ruidosas.

O una mezcla de ambas, en la que determinados registros son reetiquetados y otros son eliminados según un criterio de diseño del filtro (como lo es la confianza de que una instancia es ruidosa).

Aunque la hipótesis subyacente sea que, al reducir la presencia de ruido, el modelo podrá aprender fronteras de decisión más representativas y generalizar mejor sobre datos no vistos, el filtrado también debe aplicarse con cautela. Eliminar demasiadas instancias puede provocar pérdida de información útil, especialmente en regiones cercanas a las fronteras de decisión, donde los ejemplos pueden parecer ambiguos sin ser realmente erróneos. Del mismo modo, un filtro demasiado conservador puede dejar en el conjunto de entrenamiento una cantidad significativa de ruido, reduciendo su utilidad. Por ello, tal y como discutiremos posteriormente, al evaluar este tipo de métodos no basta con medir únicamente la mejora final obtenida en fases de clasificación, sino que también resulta importante analizar aspectos como el porcentaje de instancias eliminadas, la capacidad del filtro para identificar correctamente ejemplos ruidosos y el coste computacional del proceso, entre otros.

En consecuencia, los métodos de filtrado constituyen una alternativa relevante para el tratamiento del ruido en aprendizaje automático. Su interés reside en que permiten mejorar la calidad de los datos antes del entrenamiento, manteniendo cierta independencia respecto al clasificador utilizado. Esta característica los convierte en una opción adecuada cuando se desea diseñar o evaluar herramientas generales de tratamiento de ruido, especialmente en contextos donde no se conoce de antemano la naturaleza exacta del ruido presente en los datos.

2.4.3 Taxonomía de los filtros de ruido

De cara a la posterior presentación de la biblioteca propuesta en este trabajo, presentamos a continuación una viable y aceptada clasificación de las técnicas de filtrado en:

- **Basados en distancias/vecindad/similitud:** Para cada una de las instancias a analizar toman los k vecinos más cercanos según una medida de similitud dada y comparan los

⁴Un ejemplo de esto, como se verá posteriormente, son los algoritmos que realizan el etiquetado a través del voto de múltiples clasificadores, que individualmente podrán o no etiquetar una instancia como ruidosa; pero donde lo relevante de cara a la construcción de FS y ND es el voto global final.

atributos de la primera con los de su vecindad, a partir de lo que, junto con un posible criterio (como el grado de confianza basado en la distancia o el acuerdo/similitud entre vecinos), etiquetan o no como ruidosa la instancia original.

Dentro de este tipo podríamos enmarcar algoritmos como EditedNearest Neighbors(ENN), Nearest Centroid Neighbors Edition(NCNEEdit) o Tomek Links, entre otros tantos.

- **Basados en clasificadores auxiliares:** Estos se valen de uno o más clasificadores a partir de los que estudiar de modo individual e independiente cada una de las instancias. Dado uno o un conjunto de clasificadores base (auxiliares), este tipo de filtros dividen el conjunto de datos inicial completo y crudo TR en k subconjuntos $TR_1, TR_2, \dots, TR_k$: $TR_i \cap TR_j = \emptyset \quad \forall i \neq j$ y $TR = TR_1 \cup TR_2 \cup \dots \cup TR_k$. A continuación utilizan los anteriores para entrenar varias copias de los modelos base siguiendo una estrategia específica (dependiente del tipo de filtro, pudiendo entrenar un modelo sobre cada subconjunto, un modelo sobre cada combinación de $k - 1$ subconjuntos posibles, etc). Una vez con todos los modelos entrenados, estos se emplean (siguiendo de nuevo una estrategia dependiente del tipo de filtro) para etiquetar todas y cada una de las instancias del conjunto completo.

Dentro de este grupo se enmarcarían filtros como Classification Filter (CF), Cross Validated Committees Filter (CVCF) o Iterative Partitioning Filter (IPF), entre muchos otros.

2.5 Análisis de las bibliotecas/frameworks destinados a la limpieza de datos ruidosos

Ladeando por un instante la teoría referente al ruido, en esta sección realizaremos un análisis de los frameworks y bibliotecas disponibles hasta día de hoy de cara al tratamiento y filtrado de conjuntos de datos ruidosos.

Si realizamos en Scopus un búsqueda estructurada como la que se muestra en la Figura 2.3 (en la que se piden documentos que traten sobre datos o etiquetas ruidosas a la par que sobre limpieza de datos, preprocesamiento o modelos de aprendizaje robustos)

("noisy data" OR "label noise" OR "noisy labels") AND ("robust learning" OR "data cleaning" OR "preprocessing")

Figure 2.3: Búsqueda estructurada de Scopus a partir de la cual se basa el análisis de las herramientas actuales para el filtrado de ruido.

obtendremos un total de 6 872 documentos relacionados a la búsqueda, con una evolución temporal en la producción como la mostrada en la figura 2.4, que, además de encontrarse (aunque a menor escala) alineada con lo presentado en la introducción en la figura 1.1, pone de manifiesto la relevancia actual del mismo.

Viendo lo anterior podríamos llegar a pensar que, así como existen herramientas de cara al trabajo relacionado con el machine learning en general (como lo son [scikit-learn](#) y [PyTorch](#) en el contexto de bibliotecas de Python; o [Databricks](#) y [Google Vertex AI](#) como herramientas



Figure 2.4: Evolución temporal aproximada del número de documentos relacionados con el filtrado de conjuntos de datos ruidosos entre los años 2010 y 2025 en [Scopus](#).

de más alto nivel), lo mismo es de esperar en relación con el tratamiento de conjuntos de datos ruidosos. Considerando no solo todas las revisiones realizadas en torno a la temática, sino además todas las propuestas realizadas para abordar el problema.

Sin embargo, al realizar una revisión más detallada de las herramientas disponibles, se observa que la amplia producción científica en torno al aprendizaje con datos ruidosos no se corresponde necesariamente con la existencia de un ecosistema software igual de amplio y homogéneo. Esto es especialmente visible cuando se restringe la búsqueda a bibliotecas reutilizables, de propósito general, orientadas específicamente al filtrado de ruido en problemas de clasificación supervisada. Por un lado se tiene que muchas de las propuestas existentes en la literatura se acompañan únicamente de implementaciones asociadas a un trabajo concreto, no siempre mantenidas a largo plazo ni diseñadas como bibliotecas extensibles y compatibles con estándares como `scikit-learn`. Otras, en cambio, aparecen integradas dentro de herramientas más generales de minería de datos, preprocesamiento o aprendizaje automático, sin constituir una solución centrada exclusivamente en el filtrado de instancias ruidosas, limitando usualmente su dominio de aplicación al mismo framework o biblioteca propuesta.

En este sentido, las herramientas más próximas al objetivo de este trabajo pueden agruparse en dos bloques. Por un lado, se encuentran aquellas bibliotecas directamente orientadas a la detección, filtrado o tratamiento de ruido de etiqueta, entre las que destacan [NoiseFiltersR](#), [scikit-clean](#) y [cleanlab](#). Por otro lado, existen herramientas parcialmente relacionadas, como [KEEL](#), [imbalanced-learn](#) o [cleanvision](#), que incorporan técnicas útiles para la limpieza de datos o para problemas cercanos, pero cuyo objetivo principal no es necesariamente proporcionar una biblioteca específica de filtros de ruido para datos tabulares supervisados que además sea compatible con otros módulos ampliamente aceptados y usados.

Una de las propuestas más relevantes desde el punto de vista taxonómico es *NoiseFiltersR*, un paquete desarrollado en R para el preprocesamiento de ruido de etiqueta en tareas de clasificación [20]. Su importancia reside en que no se limita a implementar un filtro concreto, sino que organiza una colección amplia de métodos clásicos y de estado del arte siguiendo una clasificación explícita. En particular, el paquete distingue los filtros según el modo en que identifican el ruido, separando métodos basados en ensambles de clasificadores, métodos basados en similitud o vecindad y métodos basados en medidas de complejidad de datos. Además, diferencia también entre estrategias de eliminación, reparación o enfoques híbridos, dependiendo de si las instancias sospechosas son descartadas, reetiquetadas o tratadas de forma condicionada a la confianza del filtro.

Desde esta perspectiva, *NoiseFiltersR* constituye una referencia especialmente valiosa para este trabajo, ya que materializa en forma de software una parte importante de la taxonomía clásica del filtrado de ruido. Entre los filtros implementados se encuentran métodos basados en vecindad, como *Edited Nearest Neighbors* (ENN), *Repeated Nearest Neighbors* (RNN), *Tomek Links*, *AENN*, *CNN* o distintas variantes de *DROP*; así como filtros basados en clasificadores o ensambles, como *Classification Filter*, *Ensemble Filter*, *Cross-Validated Committees Filter* (CVCF) o *Iterative Partitioning Filter* (IPF), entre otros. No obstante, pese a su valor académico, su principal limitación para el objetivo de este proyecto es doble: por un lado, se encuentra desarrollado en R, mientras que el ecosistema de aprendizaje automático más extendido actualmente en entornos aplicados tiende a concentrarse en Python; por otro, su estado de mantenimiento parece limitado, habiendo sido archivado en CRAN en 2022.

Por otro lado, dentro del ecosistema Python una de las bibliotecas más próximas en espíritu a una herramienta específica para ruido de etiqueta es *scikit-clean*. Esta se presenta como una biblioteca orientada a la clasificación en presencia de etiquetas ruidosas, con implementaciones de distintos algoritmos, herramientas para simular ruido artificial, mecanismos para evaluar métodos robustos y una API compatible con *scikit-learn* [25]. Conceptualmente, su diseño resulta especialmente interesante porque separa entre detectores de ruido y manejadores del ruido. Es decir, por un lado se estima qué instancias son potencialmente problemáticas y, por otro, se decide qué hacer con dicha información, ya sea filtrando instancias, ponderándolas o entrenando modelos robustos.

Esta separación entre detección y tratamiento encaja de forma natural con la discusión planteada anteriormente en esta memoria, donde se distinguía entre el conjunto de entrenamiento original TR , el conjunto filtrado FS y el conjunto de instancias detectadas como ruidosas ND . Además, *scikit-clean* incorpora mecanismos de simulación de ruido y Pipelines diseñados para evitar fugas de información (o *data leakages*) durante la evaluación experimental. Esta característica resulta relevante, ya que en el estudio de filtros de ruido es frecuente trabajar con ruido sintético introducido de forma controlada (únicamente sobre el conjunto de entrenamiento, y no el de validación), lo que exige asegurar que la generación de ruido, la detección de instancias problemáticas y la validación del modelo final se realicen siguiendo un protocolo experimental consistente. No obstante, aunque *scikit-clean* es probablemente la propuesta Python más cercana a una biblioteca de filtrado de ruido, su actividad pública parece concentrarse principalmente en torno a 2020, por lo que no puede considerarse una solución mantenida dentro del ecosistema actual. A lo anterior se suma que carece de implementaciones de algoritmos clásicos y más reconocidos, como los ofreci-

dos por NoiseFiltersR.

Otra herramienta relevante es *cleanlab*, una biblioteca Python centrada en la detección automática de problemas en conjuntos de datos y en el paradigma conocido como *data-centric AI* [21, 5]. A diferencia de las propuestas anteriores, *cleanlab* no se presenta como una colección de filtros clásicos de ruido de instancia, sino como una herramienta general para identificar errores de etiqueta, valores atípicos, duplicados, instancias fuera de distribución y otros problemas de calidad de datos en general. En el caso de clasificación, la biblioteca permite detectar posibles errores de etiqueta a partir de las probabilidades predichas por un modelo, asignando a cada instancia una puntuación de calidad de etiqueta y permitiendo entrenar modelos de forma más robusta mediante la clase *CleanLearning*.

La principal fortaleza de *cleanlab* se encuentra en su madurez como paquete Python moderno, su documentación, su compatibilidad con distintos tipos de datos y su orientación práctica hacia la auditoría de datasets reales. Sin embargo, precisamente por su carácter generalista, su objetivo no es reproducir de forma exhaustiva la familia clásica de filtros de ruido estudiados en la literatura de preprocesamiento. En consecuencia, *cleanlab* representa más bien una herramienta complementaria: cubre de forma sólida la detección moderna de problemas de datos y etiquetas, pero no sustituye directamente a una biblioteca cuyo objetivo sea reunir, bajo una API homogénea, filtros de ruido basados en vecindad, clasificadores auxiliares, ensembles, particiones y estrategias iterativas.

Junto a estas propuestas directamente relacionadas y unidas a un lenguaje de programación específico, existen también herramientas generalistas que incluyen funcionalidades relevantes para el problema tratado. Un caso clásico es *KEEL*, un entorno Java de minería de datos desarrollado con fines de investigación y experimentación, que incorpora algoritmos de aprendizaje, técnicas de preprocesamiento, selección de instancias, selección de atributos, imputación de valores perdidos y metodologías estadísticas para la comparación experimental [1]. Sin embargo, *KEEL* no responde al modelo de biblioteca ligera e interoperable con el ecosistema Python actual, sino al de una plataforma académica generalista y cerrada.

De manera indirecta merece atención también *imbalanced-learn*, una biblioteca Python ampliamente utilizada para abordar problemas de desequilibrio de clases [14]. Aunque su objetivo principal no es el tratamiento de ruido, incluye varios métodos de submuestreo y limpieza de instancias que se solapan con filtros clásicos de ruido o que pueden emplearse de forma equivalente en determinados contextos. Entre ellos se encuentran *Tomek Links*, *Edited Nearest Neighbours*, *Repeated Edited Nearest Neighbours*, *All KNN*, *Neighbourhood Cleaning Rule* o *Instance Hardness Threshold*. Esta biblioteca constituye, por tanto, una fuente moderna y bien mantenida de implementaciones relacionadas con filtros basados en vecindad, pero su marco conceptual se orienta al balanceo de clases y no específicamente a la detección de etiquetas ruidosas.

Finalmente, pueden citarse herramientas como *cleanvision*, también perteneciente al ecosistema de *data-centric AI*, pero centrada en la detección de problemas en conjuntos de imágenes [6]. Esta biblioteca permite identificar imágenes borrosas, sobreexpuestas, subexpuestas, duplicadas o con baja información, entre otros problemas. Aunque no está orientada al ruido de etiqueta ni a datos tabulares supervisados, resulta relevante porque evidencia una tendencia reciente hacia herramientas especializadas en la auditoría automática de datasets.

En este sentido, *cleanvision* puede entenderse como una propuesta cercana en filosofía, pero aplicada a un tipo distinto de ruido: no el ruido de clase o de instancia en datos tabulares, sino problemas de calidad visual en conjuntos de imágenes.

La revisión anterior permite extraer varias conclusiones relevantes para la propuesta de este trabajo. En primer lugar, sí existen implementaciones previas de filtros de ruido y herramientas orientadas a la detección de problemas en datos, pero el ecosistema se encuentra fragmentado. Algunas propuestas, como *NoiseFiltersR*, ofrecen una cobertura taxonómica amplia, pero no pertenecen al ecosistema Python ni parecen encontrarse activamente mantenidas. Otras, como *scikit-clean*, se aproximan al objetivo de una biblioteca Python específica para ruido de etiqueta, pero su desarrollo público parece limitado. Por su parte, *cleanlab* representa una solución moderna y potente para detección de errores de etiqueta y otros problemas de calidad, aunque no como colección exhaustiva de filtros clásicos. Finalmente, herramientas como *imbalanced-learn* o *KEEL* contienen técnicas reutilizables, pero dentro de marcos conceptuales más amplios o incluso limitados al mismo programa.

En consecuencia, parece existir un espacio razonable para una biblioteca Python centrada específicamente en el filtrado de ruido en datos supervisados, con una API homogénea, documentación orientada al usuario, integración con el ecosistema *scikit-learn*, soporte para filtros basados en vecindad y clasificadores auxiliares, y mecanismos de evaluación reproducible. Este hueco resulta especialmente claro si se busca una herramienta que combine la claridad taxonómica de *NoiseFiltersR*, la filosofía modular de *scikit-clean*, la ergonomía de las bibliotecas modernas de Python y la posibilidad de incorporar clasificadores tabulares de estado del arte como parte de filtros basados en modelos auxiliares. Desde esta perspectiva, la propuesta desarrollada en este trabajo no pretende competir con herramientas generalistas de auditoría de datos como *cleanlab*, sino cubrir un ámbito más específico: el diseño de una biblioteca enfocada en filtros de ruido para clasificación supervisada, especialmente en datos tabulares, que permita comparar métodos clásicos y nuevas variantes bajo una interfaz común y estandarizada.

De tal modo, con este trabajo aspiramos a mezclar las ventajas de alternativas como *NoiseFiltersR*, destacable por su clara taxonomía, división y riqueza de algoritmos de filtrado de conjuntos de datos ruidosos; junto con la base en python y arquitectura basada en estándares consolidados de alternativas como *scikitclean* y *cleanlab*.

3 Sobre TabPFN y nuestra propuesta como filtro de ruido

3.1 Sobre TabPFN

3.1.1 Sobre la aplicación del aprendizaje profundo en datos tabulares

Pese a la demostrada poderosa capacidad de modelos predictivos basados en redes neuronales artificiales en contextos como la visión por computador o el lenguaje natural, en los que se han establecido como estado del arte, estos, quizás por su mayor coste de entrenamiento tanto temporal como computacional, no disfrutaban del mismo escenario cuando se trata de analizar datos tabulares, donde hasta hace muy poco eran alternativas relativamente modernas basadas en técnicas tipo *gradient boosting* (como XGBoost o CatBoost) las que lideraban tales tipos de problemas.

No es que que las ANN (*Artificial Neural Networks*) no estén listas para desempeñar labores de predicción en escenarios tabulares, sino que su coste, frente a las alternativas antes mencionadas, las cuales suelen desempeñarse considerablemente bien en la mayoría de escenarios, es muy alto.

3.1.2 PFNs

No obstante existe una rama dentro del campo del aprendizaje profundo (o *Deep Learning*) que investiga las denominadas en inglés como PFNs, ante *Prior-data Fitted Networks*. Es decir, redes ajustadas a un *prior* (o hipótesis a priori).

Este tipo de redes neuronales, en lugar de seguir el esquema clásico de ajuste único e individual a cada conjunto de datos a analizar mediante la constante retropropagación de los errores obtenidos en la red ajustando los pesos al realizar predicciones, durante un determinado número de épocas (*epochs*), sobre una cantidad específica de lotes (*batches*) de los datos de entrenamiento ¹; las PFNs proponen una dinámica diferente según la cual son preentrenadas, no con un conjunto de entrenamiento formado por el par atributos de entrada-etiqueta de salida (X_{train}, y_{train}) , sino con la dupla $\{(X_{train}, y_{train}), X_{test}\}$, junto con el conjunto de etiquetas de salida a predecir Y_{test} , donde, siguiendo reglas bayesianas, permite a la red estimar, a partir de unos *priors* la distribución de probabilidad ajustada a los datos de entrada.

Formalmente, este tipo de redes son preentrenadas con duplas sintéticas de la forma $\{D_{train}, x_{test}\}$, donde D_{train} conforma un conjunto de datos de entrenamiento completo y x_{test} el conjunto de datos de test, asociado al anterior, y sobre el que realizar la inferencia.

¹Con esto no tratamos de decir que no se entrenan mediante retropropagación, pues sí lo hacen; sino que no lo hacen de manera individual para adaptarse a un único dataset, sino a un conjunto de los mismos generados a partir de un *prior*

Las PFNs son así entrenadas con numerosas duplas generadas de manera aleatoria siguiendo, y aquí está el punto más importante, una serie de *priors*. Este tipo de *priors* pueden ser de la forma:

- D_{train} puede tener ruido.
- D_{train} puede tener datos faltantes.
- Pueden existir redundancias en determinados atributos de D_{train} .
- La etiqueta de salida puede estar condicionada a interacciones entre 2 o más atributos de entrada en D_{train} .
- ...

De tal modo, este tipo de redes, en lugar de aprender de manera específica un dataset mediante la retropropagación de la acumulación del error sobre predicciones individuales (como clásicamente hacen las redes neuronales artificiales), aprenden ajustando las distribuciones de probabilidad condicionadas a los *priors* con las que son concebidas.

De manera más comprensible, las PFNs, dado un conjunto de entrenamiento D_{train} y unos atributos en tests X_{test} sobre los que realizar la inferencia y_{test} , aprenden a calcular:

$$p(y_{test} = y | (D_{train}, X_{test}), \theta)$$

Es decir, aprenden a computar cuál es la probabilidad de $y_{test} = y$ condicionado a los *priors* θ con los que la red es previamente entrenada en modo offline, junto con el conjunto de entrenamiento D_{train} .

En tal sentido, aunque la arquitectura de la red no deje de ser relevante, son los *priors* a partir de los que se generan los datos los que toman el papel más relevante. Pues es a partir de estos que se trata de diseñar una red posteriormente generalizable de cara al mundo real, que idealmente debe reflejar dichos *priors*.

3.1.3 TabPFN

En la anterior exposición no hemos limitado a las tuplas (D_{train}, X_{test}) a pertenecer a ningún tipo de dato específico y cerrado. La propuesta de TabPFN se centra principalmente en subrayar dichas tuplas dentro del contexto de tabulares, generando estas a partir de *priors* innovadores, así como realizando pequeñas modificaciones de eficiencia y atención (al estar basado en modelos tipo transformer) a lo desarrollado previamente.

Sin entrar excesivamente en detalle, Hollman et al. [11] limitan el tipo de redes antes presentadas (PFNs) al dominio de los datos tabulares, entrenando un modelo tipo transformer junto con tal tipo de redes sobre una gran cantidad de tareas sintéticas a partir de una serie de *priors* que parten de premisas como la simplicidad, las interacciones de los atributos en varios sentidos, ... *priors* que han demostrado posteriormente un desempeño excelente de la red en diverso número de escenarios, incluso frente al anterior estado del arte en el entrenamiento sobre conjuntos de datos tabulares.

Al tratarse este tipo de redes, las PFNs y, más específicamente, TabPFN, de redes preentrenadas sobre un conjunto masivo de escenarios diferentes (construidos a partir de los *prior*) tienen un valor adicional y competitivo frente a la metodología de entrenamiento-validación clásica, en la cual se realizan esquemas de validaciones cruzadas con diferentes hiperparametrizaciones, a partir de lo que obtener una estimación puntual del potencialmente modelo con mejor hiperparámetros. El tipo de redes aquí descrito no requiere de ninguna hiperparametrización referente al mismo modelo, pues, como se ha dicho antes, a la de resolver un problema en el que se parte de una tupla (D_{train}, X_{test}) estas no se entrenan primero sobre D_{train} y luego infieren sobre X_{test} ; sino que, dados los priors con los que la red ha sido previamente entrenada en modo offline, junto con la dupla anterior, computa, en una sola pasada hacia delante (*forward*), la probabilidad condicionada de todo el conjunto sobre el espacio inferencia.

Sumado a que la inferencia, para conjuntos de datos de tamaño pequeño/medio, se realiza en segundos, así como con resultados estado del arte, esto último libra al usuario de necesitar preocuparse por diseñar espacios de hiperparámetros y modelos sobre los que realizar validaciones cruzadas de modo iterativo. Ello, pues, no tiene solo un valor en tanto en cuanto reduce el tiempo necesario para plantear las configuraciones a testear, así como incluso el conocimiento experto necesario para realizar lo anterior; sino que, además, tal y como se ha expuesto antes, al realizar la inferencia en una sola pasada, libra al investigador de todo el tiempo de ejecución requerido para explorar el espacio de configuraciones a estudiar.

3.2 Propuesta: TabPFN sobre filtros basados en clasificadores

3.2.1 Motivación

Dicho lo anterior, resulta natural preguntarse si, del mismo modo que clasificadores clásicos como C4.5 [22], ampliamente empleados e influyentes en el ámbito de la clasificación supervisada [31], han sido utilizados como clasificadores auxiliares en filtros de ruido basados en clasificadores, como los filtros por consenso o mayoría propuestos por Brodley y Friedl [4] y los esquemas basados en comités validados de forma cruzada de Verbaeten y Van Assche [28], un clasificador tabular moderno como TabPFN podría desempeñar un papel análogo dentro de un filtro de ruido de clase.

Remarcamos nuevamente el interés de lo anterior no solo por el buen desempeño que el modelo antes expuesto ha demostrado tener en una amplia ristra de tareas tabulares; sino porque, además, libra al investigador de la parte encargada de hiperparametrizar el clasificador auxiliar empleado en el filtro basado en modelos de clasificación.

3.2.2 Objetivos

Consecuentemente este estudio se propone, de forma ordenada, los siguiente tres objetivos:

1. **Evaluar el rendimiento de TabPFN como clasificador base de filtrado de ruido.** Se estudiará si sustituir el clasificador auxiliar de los filtros ClassificationFilter, IPF y

3 Sobre TabPFN y nuestra propuesta como filtro de ruido

CVCF por TabPFN mejora la capacidad de detectar ruido de clase y el rendimiento final del modelo entrenado sobre los datos filtrados.

2. **Analizar la explicabilidad del proceso de filtrado.** Una vez discutido el desempeño del filtro se estudiará si la técnica de explicabilidad *SHAP* permite interpretar qué atributos están influyendo en que una instancia sea considerada ruidosa.
3. **Explorar la generación automática de explicaciones para no expertos.** Si las explicaciones obtenidas muestran información útil y coherente, se estudiará la posibilidad de emplear un LLM pequeño para transformar dichas salidas en explicaciones en lenguaje natural, orientadas a usuarios no expertos.

4 Descripción de la biblioteca

4.1 Sobre la arquitectura y funcionamiento

Una [búsqueda en GitHub](#) del concepto "Machine Learning", filtrando por aquellos repositorios relacionados al lenguaje de programación *python* y ordenada descendientemente según el número de estrellas asociado, nos mostrará en el cuarto puesto la librería [scikit-learn](#), superada solo por 2 repositorios enfocados en grandes modelos de lenguaje (o *LLMs*, por su nomenclatura inglesa) y otro que reúne grandes conjuntos de módulos y librerías de diferentes lenguajes de programación y relacionados al aprendizaje automático. En vista de lo anterior, e inspirados por la librería [imbalanced-learn](#), hemos considerado de interés capital diseñar la librería siguiendo la misma lógica que, y haciéndola compatible con, la archirreconocida scikit-learn.

Esta se asienta así sobre dos tipos de objetos base:

- **Generadores de ruido (`noislearn.noisers`):** encargados de alterar los datos de entrada con un tipo de ruido dado por el generador específico seleccionando.
- **Filtros de ruido (`noislearn.filters`):** encargados de detectar y remover o reetiquetar instancias detectadas como ruidosas según el filtro previamente seleccionado.

Ambos tipos de objetos desempeñan su labor a partir del método `.fit_resample(X,y)`¹, el cual, dados los atributos de entrada en *X* (como un array bidimensional de numpy o un dataframe de pandas) y la variable objetivo en *y* (siguiendo una estructura como la anterior), devuelve los mismos dos elementos modificados según el objeto (generador de ruido o filtro) empleado.

Para el uso de los objetos anteriores dentro de un pipeline se requiere emplear el mismo objeto Pipeline definido en la biblioteca `noislearn` (o alternativamente [el de la biblioteca imbalanced-learn](#), que es exactamente el mismo)².

A continuación realizaremos una breve introducción a las técnicas de generación y filtrado de ruido implementadas en la librería a fecha de la entrega de este trabajo.

4.2 Implementaciones de la biblioteca

A continuación introducimos y resumimos brevemente las implementaciones realizadas en la biblioteca. Se presentará la teoría tras las mismas, pudiendo encontrar cómo apli-

¹Cabe resaltar que, como hace la librería `imbalanced-learn`, se ha decidido usar este método en lugar del usualmente usado para transformar los datos `TransformerMixin.fit_transform(X,y,...)` porque, tal y como indica la documentación de `scikit-learn`, los transformadores deben siempre devolver el mismo número de instancias de salida que las de entrada (algo que no cabría esperar de los filtros de ruido realizaran).

²La razón tras esta es la expuesta en la previa nota a pie de página, pues el objeto Pipeline de `scikit-learn` está preparado para trabajar solo con objetos que implementen la función `.fit_transform()`

carlas computacionalmente en la correspondiente documentación online de la API, desde noislearn.org.

4.2.1 Métodos de generación de ruido

Los métodos de generación de ruido de clase implementados en la biblioteca en el submódulo `noisers` son:

URLFNoise

Este método genera ruido independiente de la clase. Formalmente, sea un conjunto de etiquetas $\mathcal{Y} = \{y_1, y_2, \dots, y_n\}$ donde cada etiqueta pertenece a una de K clases posibles ($y_i \in \{c_1, c_2, \dots, c_k\}$), dado un nivel global de ruido $\rho \in [0, 1]$, se selecciona aleatoriamente una fracción ρ de las etiquetas y_i del conjunto de datos para posteriormente alterar cada una de las seleccionadas reemplazándola aleatoriamente por una clase $\tilde{y}_i$ distinta entre las disponibles.

Se tiene así que si al elegir la nueva clase uniformemente entre las $K - 1$, la probabilidad de que la etiqueta asociada a y_i pase de ser $y_i = c_j$ a ser $\tilde{y}_i = c_k$, viene dada por:

$$P(\tilde{y}_i = c_j \mid y_i = c_k) = \begin{cases} 0, & \text{si } c_j = c_k, \\ \frac{1}{K-1}, & \text{si } c_j \neq c_k. \end{cases}$$

De modo que, en general, la probabilidad total de que una etiqueta y_i sea alterada es igual a ρ , independientemente de la clase original asociada (todas las clases tienen la misma probabilidad de ser alteradas).

NARNoise

Este método genera ruido dependiente de la clase asociada a la variable de salida y_i . Formalmente, sea el mismo conjunto de etiquetas $\mathcal{Y}$ antes presentado, a diferencia de URLFNoise, en este caso cada clase c_k tiene asociada una probabilidad específica de alteración $\rho_{c_k} \in [0, 1]$. Por tanto, la probabilidad de que una etiqueta sea modificada depende de la clase original a la que pertenezca.

Así cada etiqueta $y_i = c_k$ será alterada con una probabilidad ρ_{c_k} . En caso de serlo, se reemplazará aleatoriamente por una clase distinta (de c_k) entre las $K - 1$ clases restantes siguiendo el mismo esquema de aleatoriedad antes presentado (es decir, se selecciona una clase diferente con probabilidades iguales entre las disponibles).

De modo que, en general, la probabilidad total de que una etiqueta $y_i = c_k$ sea alterada es igual a ρ_{c_k} , dependiendo de la clase original asociada. Por ello, algunas clases pueden verse más afectadas por el ruido que otras.

4.2.2 Métodos de filtrado de ruido de clase

Los filtros disponibles en la biblioteca son los siguientes:

ENNFilter

Implementa *Edited Nearest Neighbour* (ENN) y *Modified Edited Nearest Neighbour* (MENN).

El primero parte del conjunto de entrenamiento completo TR y elimina cada instancia cuya clase no coincide con la mayoría de sus k vecinos más cercanos.

El segundo añade a la lista de vecinos con que compara todos aquellos a una misma distancia que el más lejano.

ENNProbFilter

Implementa *ENNProbabilistic* y *ENNProbabilisticThreshold*, dos variantes probabilísticas de ENN. Al esquema de decisión de ENN añaden una probabilidad ponderada sobre la distancia de los k vecinos más cercanos. Es decir, pesa la etiqueta de cada uno de los vecinos según la distancia a estos.

El segundo utiliza adicionalmente un umbral que impone un mínimo nivel de acuerdo entre los vecinos (lo que, tras normalizar los pesos anteriores, es equivalente a poner un umbral sobre la etiqueta mayoritariamente predicha a partir del vecindario).

AllKNN

extensión de ENN en la que se aplica la regla de vecino más cercano de forma progresiva incrementando el número de vecinos considerados desde 1 hasta un valor k dado.

NCNEdit

Implementa *Nearest Centroid Neighbor Edition*. Este se diferencia de las técnicas anteriores al definir el vecindario de cada instancia teniendo en cuenta la proximidad de los prototipos junto con su distribución alrededor del ejemplo mediante vecinos centroidales. Es decir, para cada instancia x_i a filtrar, de cara a construir su vecindario de k vecinos (llamémoslo $N = \{n_1, n_2, \dots, n_k\}$) comienza (en el paso $t = 1$) buscando su vecino más cercano (n_1) para a continuación añadir, de uno en uno, los $k - 1$ vecinos restantes ($n_2, \dots, n_k$), iterativamente (en k pasos, desde $t = 1$ hasta $t = k$), cumpliendo que en cada paso $t = j$ el centroide definido por los $j - 1$ vecinos previamente seleccionados (sean estos $n_1, n_2, \dots, n_{j-1}$) minimice la distancia a la instancia x_i a filtrar.

MultiEditFilter

Método iterativo que divide el conjunto de entrenamiento en bloques estratificados y aplica una regla kNN entre bloques sucesivos para eliminar las instancias mal clasificadas.

Comienza separando de forma aleatoria y estratificada los datos iniciales (que conforma TR) en b subconjuntos indexados $R_i : i = 1, 2, \dots, b$, para predecir las etiquetas de cada conjunto R_i a partir de un KNN entrenado en el conjunto que le sucede (siendo este $R_{(i-1) \bmod(b)}$). Las instancias incorrectamente clasificadas al seguir el esquema anterior son consecuentemente son filtradas y eliminadas del conjunto de datos FS .

Este proceso se repite iterativamente, realizando en cada paso diferentes particionamientos

REPASAR EL QUE
Adicionalmente no emplea un solo valor k para filtrar sino ...Furthermore, MENN works with a prefixed number of pairs (k, k') . k is employed as the number of neighbors used to perform the editing process, and k' is employed to validate the edited set FS obtained. The best pair found is employed as the final reference set. If two or more sets are found to be optimal, then both are used in the classification of the test instances. A majority rule is used to decide the output of the classifier in this case.

IMPLEMENTAR ESTE DE VERDAD

de los datos, hasta que se cumpla una condición de parada dada (como un porcentaje de instancias filtrado, un número máximo de iteraciones, ...).

Implementar esto de verdad

TomekLinks

una conexión de Tomek se define como un par de instancias (x_1, x_2) de clases diferentes tal que para cada una de ellas no existe otra instancia x_3 de una clase diferente a una menor distancia. Matemáticamente $\nexists x_3 : d(x_1, x_2) > d(x_1, x_3), d(x_2, x_3)$. Este tipo de pares de puntos suelen encontrarse en fronteras de decisión o en regiones mayoritariamente pobladas por una de las dos clases (pudiendo con ello, aunque no determinadamente, suponer ruido). De modo que tras detectar cada conexión de Tomek del conjunto de datos, todo par es removido del mismo.

ClassificationFilter

Filtro basado en un clasificador arbitrario f y validación cruzada estratificada.

Comienza particionando los datos de manera estratificada y aleatoria en b subconjuntos (o *folds*) $F_1, F_2, \dots, F_b$ tal que $F_i \cup F_j = \emptyset \forall i \neq j$ y $TR = F_1 \cup F_2 \cup \dots \cup F_b$. A continuación, para cada particionamiento F_i se entrena un algoritmo de clasificación f_i (con la misma configuración/hiperparametrización del original f) en los $b - 1$ restantes, obteniendo con ello b clasificadores $(f_1, f_2, \dots, f_b)$, los cuales se emplearán cada uno por separado para realizar predicciones sobre subconjunto en el que no han sido entrenados, etiquetando como ruidosas aquellas instancias incorrectamente clasificadas. Es decir, las instancias del subconjunto F_1 son etiquetadas como potencialmente ruidosas por el clasificador f_1 entrenado sobre los $b - 1$ subconjuntos restantes, es decir sobre $TR/F_1 = F_2 \cup F_3 \cup \dots \cup F_b$. Para procesar todo el conjunto de entrenamiento TR se procede del mismo modo con las $b - 1$ particiones restantes.

EnsembleFiltering

Generaliza la propuesta anterior entrenando, no solo un clasificador por partición, sino un ensemble de clasificadores a partir del que etiquetar las instancias como potencialmente ruidosas mediante un voto (por mayoría, consenso o umbral).

Es decir, dada una lista de l clasificadores auxiliares $f_1, f_2, \dots, f_l$ ³, el conjunto de entrada TR es nuevamente dividido en b particiones $F_1, \dots, F_b$, de modo que para cada una se entrena un *ensemble* de clasificadores en las otras $b - 1$ particiones, para posteriormente clasificar como ruidosas o no (según un tipo de voto preseleccionado) las instancias haciendo uso del ensemble de clasificadores sobre el que no ha sido entrenada.

Es decir, las instancias de cada partición F_i son filtradas a partir del voto realizado con las predicciones del ensemble de clasificadores $(f_{1i}, f_{2i}, \dots, f_{li})$ previamente entrenado sobre las $b - 1$ particiones restantes, es decir, sobre el conjunto TR/F_i .

CVCFFilter

Implementa *Cross-Validated Committees Filter*.

Siguiendo una propuesta muy similar a la anterior, en lugar de usar cada ensemble de clasi-

³Los cuales, para que la propuesta tenga algún valor diferencial a la anterior, basta con que tengan hiperparametrizaciones diferentes, aunque usualmente se empleen en lugar de lo anterior diferentes tipos de clasificadores.

ficadores $(f_{1i}, f_{2i}, \dots, f_{li})$ para realizar predicciones únicamente sobre la partición F_i sobre la que no ha sido entrenado, todos los ensembles entrenados se emplean para etiquetar (de nuevo, mediante un tipo de voto previamente seleccionado) como ruidosas o no todas las instancias del conjunto de entrada.

IterativePartitioningFilter

Realiza un filtrado iterativo basado en particiones estratificadas y votación entre modelos. Comienza particionando los datos de entrada TR en b subconjuntos $F_1, \dots, F_b$, entrenando sobre cada uno de ellos un mismo clasificador f . Posteriormente se emplean los b clasificadores obtenidos en el paso anterior para etiquetar como ruidosas las instancias de todo el conjunto de datos, removiendo aquellas incorrectamente predichas. El proceso es repetido con el conjunto restante hasta cumplir una condición de parada dada.

A diferencia de los filtros anteriores, en el que para cada partición F_i se entrenaba un clasificador (o ensemble de los mismos) sobre el conjunto de instancias restante TR/F_i , en este caso cada clasificador f_i (a aplicar en el posterior filtrado de instancias) se entrena únicamente sobre el subconjunto F_i .

INFFCFilter

Implementa *Iterative class Noise Filter based on the Fusion of Classifiers*, un filtro iterativo que combina varios clasificadores auxiliares para detectar instancias potencialmente ruidosas.

En cada iteración $t = j$, una selección de clasificadores (conformando un ensemble) se entrenan sobre el conjunto actual $FS(t = j)$ y sus predicciones se fusionan mediante una regla de voto preseleccionada, etiquetando como candidatas a ruido aquellas instancias cuya clase predicha no coincide con la etiqueta original.

Posteriormente, sobre las instancias candidatas se calcula una puntuación de ruido que mide la evidencia de que estén mal etiquetadas, eliminando únicamente aquellas cuya puntuación supera un umbral predefinido. El proceso se repite sobre el conjunto filtrado hasta que se alcanza una condición de parada dada.

Es decir, la técnica parte en el instante $t = 1$ del conjunto $FS_1 = TR$, sobre el que entrena una serie de clasificadores que utilizará para marcar como ruidosas un conjunto de instancias ND_1 . A continuación, en el paso $t = 2$, repetirá el proceso anterior partiendo esta vez del conjunto $F_2 = FS_1/ND_1$, a partir de las que etiquetará como ruidosas un conjunto de registros ND_2 . Seguido se realizará lo mismo, esta vez a partir del conjunto $FS_3 = FS_2/ND_2 = TR/(ND_1 \cup ND_2)$... repitiendo dicho proceso hasta cumplir una condición de parada.

Contrariamente a la mayoría de algoritmos anteriores, este realiza solo un entrenamiento por clasificador en cada iteración $t = j$, sobre el conjunto de instancias disponibles aún no filtradas FS_j ; en lugar de entrenar varios sobre diferentes particiones disjuntas de los datos.

Confirmar esta segunda parte de la implementación

CNCNOS

Implementa *Classification Noise Cleaning with NOise Score*. Esta técnica reúne el poder de los filtros de ruido para etiquetar como sospechas instancias potencialmente ruidosas. Para ello

sigue en orden el siguiente esquema:

1. Dado el conjunto inicial de entrenamiento TR, somete el mismo a un ensembles de filtros (de ruido de clase) para conformar un conjunto N_F a partir de las instancias etiquetadas como potencialmente ruidosas según el voto por mayoría resultante del anterior ensemble.
2. Construido el conjunto N_F , para todas y cada una de las instancias (potencialmente ruidosas) e_i del mismo se computa su nivel de ruido $wNS(e_i)$ (por *weighted Noise Score*), dado por la ecuación 2:

$$wNS(e_i) = confidence(e_i) \cdot \overline{neighborhood}(e_i)$$

Con:

$$confidence(e_i) = \frac{1}{1 + t(e_i)^2}$$

Y:

$$\overline{neighborhood}(e_i) = \frac{\sum_{j=1}^k \overline{clean}(e_j) \cdot confidence(e_j) \cdot differentClass(e_i, e_j)}{k}$$

Tal que:

$$\overline{clean}(e_i) = \frac{\sum_{j=1}^k d_{ij} + isNoise(e_i) \cdot \left(\sum_{j=1}^k d_{ij} - \sum_{j=1}^k d_{ij} \right)}{2 \cdot \sum_{j=1}^k d_{ij}}$$

$$isNoise(e_i) = \begin{cases} 1, & \text{if } e_i \in N_F, \\ -1, & \text{if } e_i \notin N_F. \end{cases}$$

$$differentClass(e_i, e_j) = \begin{cases} 1, & \text{if } class(e_i) \neq class(e_j), \\ -1, & \text{if } class(e_i) = class(e_j). \end{cases}$$

Donde $confidence(e_i)$ mide el como de valiosa es la información obtenida del vecindario de la instancia e_i ; y $\overline{neighborhood}(e_i)$ mide el nivel de concoordancia de la intancia e_i con su vecindario, pesando la intervención de cada vecino e_j según la distancia entre ambos.

3. Una vez calculadas las puntuaciones de ruido de cada instancia, se decide si mantener (dentro del conjunto de entrenamiento), reetiquetar o eliminar la instancia e_i en el siguiente orden de prioridades:
 - a) Si $wNS(e_i) \leq 0$ se mantiene.
 - b) Si todos los filtros de clase coinciden en predecir la misma clase C_i , se reetiqueta por esta.
 - c) Si más de la mitad de los filtros coinciden en predecir la misma clase C_i y $wNS(e_i) \geq \overline{wNS}(e_i)$ se reetiqueta.
 - d) Se elimina en cualquier otro caso.

De modo que la clase asociada a una instancia potencialmente mal etiquetada será sustituida si todos los filtros coinciden en sus predecciones, o si la mayoría lo hacen y se calcula una

```

1  from sklearn.datasets import make_classification
2  from sklearn.model_selection import train_test_split
3  from noislearn.noisers import NARNoiseGenerator
4  from noislearn.filters import ENN
5  # 1. Generar un conjunto de datos de ejemplo
6  X, y = make_classification(
7      n_samples=1000,
8      n_features=20,
9      n_informative=15,
10     n_redundant=5,
11 )
12 # 2. Separar datos en entrenamiento y prueba
13 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
14     random_state=42, stratify=y)
15 # 3. Añadir ruido a las etiquetas del conjunto de entrenamiento
16 noiser = NARNoiseGenerator(noise_rate=0.2, random_state=42)
17 X_train_noisy, y_train_noisy = noiser.fit_resample(X_train, y_train)
18 # 4. Aplicar un filtro de ruido
19 filter_ = ENN()
20 X_clean, y_clean = filter_.fit_resample(X_train, y_train_noisy)

```

Código 4.1: Ejemplo básico de uso de la biblioteca noislearn.

Figure 4.1: Código con ejemplo de uso de la biblioteca.

puntuación de ruido de la instancia e_i ($wNS(e_i)$) mayor al promedio ($\overline{wNS(e_i)}$).

El proceso anterior se repetirá hasta cumplir una condición de parada (máximo número de iteraciones, hasta que no haya cambios, ...).

4.3 Ejemplo de uso

4.3.1 Uso básico de la biblioteca

En la figura 4.1 mostramos un ejemplo de uso de la biblioteca propuesta junto a este proyecto. Analizemos dicho ejemplo más en detalle.

Comenzamos con la figura 4.2 generando el conjunto de datos a filtrar a partir de la función `make_classification`. Este crea un total de 1000 pares de registros $((x_i, y_i))$ definidos por $m = 20$ atributos de entrada.

A continuación, siguiendo lo mostrado en la figura 4.3, el conjunto anterior es particionado de manera estratificada y aleatoria en los subconjuntos de entrenamiento y testeo siguiendo una división 80/20. Hasta este punto no se ha usado nada de la biblioteca `noislearn`.

Listo y particionado el conjunto de datos a emplear, tal y como muestra la figura 4.4, añadimos un 20% de ruido de clase completamente aleatorio al conjunto de entrenamiento, generando el par conjunto TR definido por el par de atributos-etiqueta $(X_{train_noisy}, y_{train_noisy})$.

Una vez con el conjunto TR listo, tras haberle introducido ruido, aplicamos el filtro `ENN` para generar un conjunto filtrado (X_{clean}, y_{clean}) a partir de los ejemplos afectados por

Confirmar que la estructura de las subceldas de código es la correcta (es decir, que está bien maquetado)

Pensar si añadir algún gráfico a todo esto o aumentar la info sobre le ejemploe

Figure 4.2: Generación de datos

```

20     from sklearn.datasets import make_classification
21     X, y = make_classification(
22         n_samples=1000,
23         n_features=20,
24         n_informative=15,
25         n_redundant=5,
26     )

```

Figure 4.3: División en entrenamiento y prueba

```

27     from sklearn.model_selection import train_test_split
28     X_train, X_test, y_train, y_test = train_test_split(
29         X, y, test_size=0.2, random_state=42, stratify=y
30     )

```

ruido ($X_{\text{train_noisy}}$, $y_{\text{train_noisy}}$).

Hecho lo anterior se dispondrá con ello de un conjunto de datos propiamente filtrado según la técnica escogida en este caso (ENN).

Lo anterior sirve de muestra de que, siguiendo la misma filosofía de la biblioteca scikit-learn, los transformadores asociados a los filtros aquí empleados no cambian individualmente todos y cada uno de los registros de entrada⁴, sino que seleccionan un subconjunto de los mismos. Es decir, al aplicar un filtro genérico de la forma $\text{filtro}(X, y)$, este no altera de ningún modo las instancias individuales del par (X, y) (donde $X = (x_1, x_2, \dots, x_n)$ e $y = (y_1, \dots, y_n)$), sino que devuelve un par nuevo generado tras seleccionar solo aquellas instancias x_i clasificadas como limpias por el filtro empleado.

Pensar si dejo
esto

4.3.2 Uso de la biblioteca con *scikit-learn*

Para el habituado a la biblioteca scikit-learn, el uso de la sintaxis `.fit_resample(...)` puede parecer antinatural, esperando en realidad emplear en tales casos `.fit_tranform(...)`.

Sin embargo, si antedemos a la [documentación](#) de dicha librería encontraremos que los objetos de tipo transformador, pese a poder retornar un menor número de columnas que las iniciales, deben siempre retornar el mismo número de filas. Es decir, estos están diseñados de modo que realicen operaciones sobre los datos de entrada que, si se desea, reduzcan el número de atributos (columnas) que los definen; pero que siempre deben de retornar el mismo número de registros (filas) original.

En tal sentido, los métodos de filtrado implementados en *nois-learn*, de modo similar a como sucede con *imbalanced-learn*, retornan un subconjunto de los datos de entrada; es

⁴Como sí sucede con objetos del tipo `StandardScaler` o `OneHotEncoder`

Figure 4.4: Adición de ruido

```

31 from noislearn.noisers import NARNoiseGenerator
32 noiser = NARNoiseGenerator(noise_rate=0.2, random_state=42)
33 X_train_noisy, y_train_noisy = noiser.fit_transform(X_train, y_train)

```

Figure 4.5: Filtrado del ruido

```

34 from noislearn.filters import ENN
35 filter_ = ENN()
36 X_clean, y_clean = filter_.fit_resample(X_train, y_train_noisy)

```

decir, retornan menos filas de las que originalmente se introducen a los transformadores (los filtros, en este caso). Es por ello que, se ha optado por usar la sintaxis `fit_resample()` en lugar de la alternativa `fit_tranform()` nativa de `scikit-learn`.

Esto, sin embargo y desgraciadamente, lo hace incompatible de modo directo con los Pipelines de `scikit-learn`. No obstante podemos evadir el anterior problema haciendo uso del objeto Pipeline implementad en la biblioteca `imbalanced-learn`, el cual está listo para valerse de transformadores basados en el método `fit_resample()`.

4.4 Módulo de explicabilidad

COMPLETAR

Como expone Lipton [15], el valor de los modelos de aprendizaje automático no está solo en su poder predictor, sino, además, en la capacidad de explicar y generalizar sus resultados.

Por ejemplo, en contextos como el diagnóstico de radiografías mediante visión por computador, sería conformista aceptar un modelo capaz de dar diagnósticos excelentes, nunca antes visto, sin saber, si quiera un experto, explicar el motivo. Es mediante técnicas de explicabilidad como los *saliency maps* o *LIME* que el experto puede además descubrir en qué regiones basa el modelo sus decisiones, pudiendo de ello emerger nuevo conocimiento (en el mismo modo en que el signo y valor de los coeficientes en una sencilla regresión lineal nos informan no solo del modo en que una variable impacta — positiva o negativamente —, sino de la intensidad con que lo hace).

En tal sentido esta biblioteca ha sido diseñada con tal hecho en mente, adaptando un módulo de explicabilidad a todos y cada uno de los filtros implementados

Valores SHAP

Antes de introducir el diseño y funcionamiento del módulo de explicabilidad antes propuesto es conveniente introducir al lector los valores Shap[17].

Históricamente el diseño de algoritmos de aprendizaje automático cada vez más flexibles y potentes ha sido realizado muchas veces a costa de una complicación en la explicabili-

dad de las decisiones que los mismos toman. Este problema resulta especialmente relevante en el contexto del filtrado de ruido de clase, ya que puede no basta únicamente con que un filtro marque una instancia como potencialmente ruidosa, sino que también es deseable conocer qué características han contribuido a dicha decisión. En este sentido, las técnicas de explicabilidad permiten complementar el proceso de filtrado proporcionando información adicional sobre el comportamiento del clasificador auxiliar.

Dentro de este conjunto de técnicas, los valores SHAP, acrónimo de *SHapley Additive ex-Planations*, constituyen una de las aproximaciones más utilizadas para explicar predicciones individuales de modelos de aprendizaje automático [17]. SHAP se basa en los valores de Shapley procedentes de la teoría de juegos cooperativos [?], donde el objetivo es repartir de forma justa la contribución total obtenida por una coalición entre los distintos jugadores que participan en ella. Trasladado al contexto del aprendizaje automático, los “jugadores” son las variables de entrada del modelo, y la “ganancia” que se reparte es la diferencia entre la predicción realizada para una instancia concreta y una predicción base o esperada.

De forma intuitiva, el valor SHAP asociado a una variable mide cuánto contribuye la misma a desplazar la predicción del modelo respecto a un valor de referencia. Si se considera un modelo predictivo f y una instancia concreta $\mathbf{x} = (x_1, x_2, \dots, x_p)$, la explicación aditiva proporcionada por SHAP puede expresarse, de manera independiente para cada una de las instancias (y esto último es de extrema relevancia, pues al contrario que métricas como la importancia de atributo — o *feature importance* — reportada por modelos como los bosques aleatorios, los valores Shap no son colectivos, sino individuales), como:

$$f(\mathbf{x}) \approx \phi_0 + \sum_{j=1}^p \phi_j,$$

donde ϕ_0 representa el valor base de la predicción (normalmente asociado a la salida media del modelo sobre un conjunto de referencia), y ϕ_j representa la contribución atribuida a la variable j para la instancia $\mathbf{x}$ (remarcando de nuevo el hecho, pues, de que dichos pesos ϕ_j , o contribuciones de cada atributo, esperablemente diferirán entre instancias). Por tanto, cada predicción individual se descompone en una suma de contribuciones atribuibles a las distintas variables de entrada.

El signo y la magnitud de cada valor SHAP permiten interpretar el efecto de cada atributo sobre la predicción. Un valor SHAP positivo indica que la característica correspondiente empuja la predicción hacia la clase o salida explicada, mientras que un valor negativo indica que la aleja de ella. A su vez, cuanto mayor sea el valor absoluto de ϕ_j , mayor será la influencia de la variable j en la predicción concreta que se está explicando. De tal modo esta interpretación local es especialmente útil cuando se desea analizar por qué una instancia concreta ha sido clasificada de una determinada manera, en lugar de limitarse a estudiar la importancia global de las variables sobre todo el conjunto de datos, pues la detección de instancias ruidosas se trata de un problema individual, no global o colectivo.

Naturalmente puede surgir en el lector la cuestión de como son computados dichos valores cuando se trabaja en escenarios de clasificación multiclase (pues téngase en cuenta que hasta el momento no hemos limitado esta técnica a ningún dominio respecto de la variable a predecir, así como tampoco del modelo a explicar). En tales casos, así como las prob-

abilidades suelen computarse por cada clase (bien comparando una con todas las demás, siguiendo un esquema OVA; o según comparaciones a pares, en un esquema OVO) los valores SHAP suelen calcularse respecto a una clase concreta. Es decir, para una instancia dada, puede estudiarse qué variables han contribuido a aumentar o disminuir la probabilidad asignada por el modelo a una determinada clase. Es decir, estos valores pueden explicar tanto el motivo por el que una instancia ha sido asignada a una determinada clase, como aquellos por los que ha sido excluida de todas y cada una de las restantes. En el caso que nos atañe, de filtrado de ruido de clase, esto permite analizar las razones por las que el clasificador auxiliar predice una etiqueta distinta de la original. Así, si una instancia con etiqueta observada y_i es clasificada por el filtro como perteneciente a otra clase $\hat{y}_i$, los valores SHAP pueden emplearse para identificar qué atributos han favorecido la predicción $\hat{y}_i$ y, por tanto, qué evidencias ha utilizado el modelo para considerar sospechosa la etiqueta original. El uso de valores SHAP permite pues enriquecer esta decisión proporcionando una justificación local basada en atributos concretos.

Por ejemplo, para una instancia marcada como ruidosa según el clasificador base a emplear en la técnica de filtrado, el análisis SHAP puede mostrar que ciertos atributos empujan fuertemente la predicción hacia una clase distinta de la etiqueta original. En tal caso, la instancia podría interpretarse como coherente con la clase predicha por el modelo y, por tanto, como candidata razonable a contener ruido de clase. Por el contrario, si las contribuciones son débiles, contradictorias o dependen de pocos atributos poco informativos, la decisión del filtro debería interpretarse con mayor cautela. De este modo, SHAP no sustituye al criterio de filtrado, pero sí proporciona una capa adicional de información que ayuda a comprender y auditar las decisiones tomadas por el filtro.

Implementación en la biblioteca

En la biblioteca propuesta se emplean así los valores SHAP con la finalidad de explicar instancias concretas que han sido (o no) marcadas como potencialmente ruidosas por el filtro. Para ello, se analizan las variables con mayor contribución absoluta sobre la predicción realizada por el clasificador auxiliar. En particular, se considera de interés identificar los k (parámetro a decidir por el analista) atributos con mayor influencia sobre la clase predicha por el modelo, junto con la confianza asociada a dicha predicción. Esta información permite construir una explicación más interpretable del tipo: la instancia ha sido marcada como sospechosa porque, aunque su etiqueta original es y_i , el modelo predice la clase $\hat{y}_i$ con una determinada confianza, principalmente debido a los valores observados en ciertos atributos relevantes.

No obstante, es importante señalar que los valores SHAP deben interpretarse como explicaciones del comportamiento del modelo, el cual puede encontrarse sesgado o no ser el idóneo de cara a la tarea a resolver, no debiendo de confundir dichos valores con relaciones causales presentes en los datos. Es decir, SHAP indica qué variables han influido en la predicción realizada por el clasificador auxiliar, pero no garantiza que dichas variables sean la causa real de que una instancia esté mal etiquetada.

En consecuencia, dentro de la propuesta desarrollada, SHAP se plantea como una herramienta de apoyo para la interpretabilidad del proceso de filtrado. Su objetivo no es de-

mostrar de forma absoluta que una instancia contiene ruido, sino proporcionar evidencias comprensibles sobre por qué el clasificador auxiliar ha considerado que su etiqueta observada no es coherente con sus atributos. Esta información resulta útil tanto para el análisis experimental del método como para una posible inspección manual posterior por parte de usuarios expertos o no expertos. Potenciaría así la extracción de conocimiento a la par que la justificación en la toma de decisiones.

Con tal objetivo en mente la API correspondiente a los filtros basados en TabPFN vienen acompañadas del método `.explain_noisy_instances(...)`, el cual retorna un análisis sobre las decisiones tomadas a la hora de clasificar como potencialmente ruidosas las instancias a filtrar. Más específicamente, el mismo método devuelve para cada una de las instancias que se deseen estudiar (hayan o no sido marcadas como potencialmente ruidosas) un diccionario como el de la figura 4.6. Este cuenta, en el orden en que se muestra, para cada una de las instancias a explicar, con el índice original, el fold en la validación cruzada subyacente al filtro, la etiqueta original, la etiqueta asignada/predicha por el filtro, la confianza de dicha predicción y el top de los k atributos con un mayor valor SHAP asociado a dicha predicción

```
{
    "sample" : e.sample,
    "fold" : e.fold,
    "true" : e.true,
    "oof_pred" : e.oof_pred,
    "confidence" : e.confidence,
    "top_k" : e.top_k
}
```

Figure 4.6: Estructura del informe a nivel de instancia retornado por los clasificadores basados en TabPFN en su etiquetado de instancias como potencialmente ruidosas.

Como se decía anteriormente, el anterior diccionario permite con ello desarrollar argumentaciones del tipo:

La instancia correspondiente al índice `e.sample`, con etiqueta original `e.true` ha sido marcada como ruidosa según el filtro empleado por corresponderle con un `e.confidence*100` por ciento de confianza la clase `e.oof_pred`. Dicha decisión se ve justificada, en orden decreciente de contribución por la lista de k pares (atributo, coeficiente_shap) `e.top_k`.

4.4.1 Ejemplo de uso

En el ejemplo anterior se muestra el uso del filtro `TabPFN_CF` sobre un conjunto de datos sintético generado con `make_classification`. En primer lugar, se construyen los datos y se dividen en entrenamiento y prueba mediante una partición estratificada, de forma análoga a como se haría en cualquier flujo de trabajo habitual con *scikit-learn*. A continuación, se instancia el filtro `TabPFN_CF`, que emplea `TabPFN` como clasificador base y realiza una validación cruzada estratificada para identificar aquellas instancias cuya etiqueta observada no coincide con la predicha fuera de muestra. El método `fit_resample` devuelve, según la acción elegida, el conjunto depurado o bien el conjunto original con las etiquetas sospechosas relabelizadas.

```

37 from sklearn.datasets import make_classification
38 from sklearn.model_selection import train_test_split
39 from filters.classifier_based.TabPFN_based.TabPFN_CF import TabPFN_CF
40
41 # 1. Generar un conjunto de datos de ejemplo
42 X, y = make_classification(
43     n_samples=1000,
44     n_features=20,
45     n_informative=15,
46     n_redundant=5,
47     random_state=42,
48 )
49
50 # 2. Separar datos en entrenamiento y prueba
51 X_train, X_test, y_train, y_test = train_test_split(
52     X, y, test_size=0.2, random_state=42, stratify=y
53 )
54
55 # 3. Crear y ajustar el filtro basado en TabPFN
56 filter_ = TabPFN_CF(cv=10, random_state=33, action="remove")
57 X_clean, y_clean = filter_.fit_resample(X_train, y_train)
58
59 # 4. Obtener un informe general del filtrado
60 report = filter_.get_filter_report()
61
62 # 5. Explicar las instancias marcadas como ruidosas
63 explanations = filter_.explain_noisy_instances(
64     noisy_only=True,
65     class_index="predicted",
66     top_k=5,
67     return_interaction_values=False,
68     return_figures=False,
69 )
70
71 # 6. Consultar la primera explicación
72 e = explanations[0]
73 print(e.sample_idx, e.fold_idx, e.true_label, e.oof_pred, e.confidence, e.top_k)

```

Código 4.2: Ejemplo básico de uso de TabPFN_CF y su módulo de explicabilidad

Figure 4.7: Código con ejemplo de uso del filtro TabPFN_CF y su módulo de explicabilidad.

Una vez ajustado el filtro, el método `get_filter_report()` permite obtener un resumen general del proceso de filtrado, incluyendo el número de muestras tratadas, el número de particiones usadas, la fracción de instancias marcadas como ruidosas y los parámetros de configuración del clasificador. Además, el método `explain_noisy_instances(...)` proporciona explicaciones locales para las instancias detectadas como ruidosas, calculando contribuciones tipo SHAP sobre la predicción fuera de muestra de cada ejemplo. En particular, el parámetro `top_k` permite recuperar los atributos con mayor influencia sobre la decisión, mientras que `class_index="predicted"` hace que la explicación se centre en la clase predicha por el filtro. De este modo, no solo se identifica qué instancias han sido consideradas sospechosas, sino también qué variables han contribuido con mayor peso a dicha decisión.

4.5 Instalación de la biblioteca

Analizar y completar esto

Para instalar la biblioteca `nois-learn` basta con...

COMPLETAR

5 Metodología de evaluación experimental

5.1 Evaluación de los filtros implementados en la biblioteca `nois-learn`

Con el objetivo de evaluar el correcto diseño y funcionamiento de los métodos de filtrado implementados en la biblioteca aquí propuesta, replicaremos los experimentos realizados por J.Luengo et al en [16], donde se compara el desempeño del, propuesto en tal artículo, limpiador de ruido CNC-Nos, frente a parte de los filtros implementados en `nois-learn`.

5.1.1 Conjuntos de datos

Se hará uso de los conjuntos de datos mostrados en la tabla 5.1, en la que se reflejan, en 2 columnas, los identificadores de los conjuntos de datos a emplear, junto con el número de instancias del que disponen (#EX), el número de atributos numéricos y nominales (#AT) y el número de clases (#CL). Para cada dataset, luego de eliminar cualquier instancia con algún atributo perdido, se estudiará el desempeño de las técnicas de filtrado tras introducir ruido de clase completamente aleatorio (NCAR). Más específicamente, se estudiarán los casos con niveles de ruido en el rango 5%(bajo), 25%(medio) y 50%(alto).

Partiendo de 31 datasets, se contará así con 93 conjuntos de datos ruidosos, sobre los que se realizarán 5 validaciones cruzadas estratificadas por la clase objetivo. Adicionalmente, por cada conjunto de datos y nivel de ruido dados, se realizarán el estudio con 5 semillas (de aleatoriedad) diferentes, dando lugar a un total de 25 ciclos de entrenamiento y validación

Table 5.1: Conjuntos de datos base a usar en la evaluación del funcionamiento de los filtros implementados en `nois-learn`.

Dataset	#Atributos (R/I/N)	#Examples	#Classes	Dataset	#Atributos (R/I/N)	#Examples	#Classes
automobile	25 (15/0/10)	150 (205)	6	penbased	16 (0/16/0)	10,992	10
car	6 (0/0/6)	1728	4	phoneme	5 (5/0/0)	5404	2
cleveland	13 (13/0/0)	297 (303)	5	pima	8 (8/0/0)	768	2
dermatology	34 (0/34/0)	358 (366)	6	ring	20 (20/0/0)	7400	2
ecoli	7 (7/0/0)	336	8	satimage	36 (0/36/0)	6435	7
german	20 (0/7/13)	1000	2	segment	19 (19/0/0)	2310	7
glass	9 (9/0/0)	214	7	shuttle	9 (0/9/0)	2,189	7
heart	13 (1/12/0)	270	2	sonar	60 (60/0/0)	208	2
ionosphere	33 (32/1/0)	351	2	spambase	57 (57/0/0)	4597	2
iris	4 (4/0/0)	150	3	splice	60 (0/0/60)	3190	3
lymphography	18 (0/3/15)	148	4	thyroid	21 (6/15/0)	7200	3
magic	10 (10/0/0)	19,020	2	vehicle	18 (0/18/0)	846	4
monk-2	6 (0/6/0)	432	2	vowel	13 (10/3/0)	990	11
newthyroid	5 (4/1/0)	215	3	wine	13 (13/0/0)	178	3
nursery	8 (0/0/8)	12,690	5	yeast	8 (8/0/0)	1484	10
page-blocks	10 (4/6/0)	5472	5				

Table 5.2: Parameter setting for the noise filters.

Name	Acronym	Ref.	Parameters
Edited nearest neighbor	ENN	[52]	k : 3, Distance function: Euclidean/HVDM
Edited nearest neighbor with thresholding	ENNTh	[52]	k : 3, Noise Threshold: 0.7, Distance function: Euclidean/HVDM
Nearest centroid neighbourhood edition	NCNEdit	[44]	k : 3
Relative neighbourhood graphs	RNG	[43]	Distance function: Euclidean/HVDM
Classification filter	CF	[16]	Γ : 10, k : 1, classifier: 1-NN
Cross-validated committees filter	CVCF	[53]	Voting scheme: consensus, Γ : 4
Ensemble filter	EF	[8]	Voting scheme: consensus, Γ : 4
Iterative-partitioning filter	IPF	[27]	Voting scheme: majority, Γ : 5, K_{IPF} : 3, D_{IPF} : 1%
Iterative class noise filter based on the fusion of classifiers	INFFC	[42]	Voting scheme: majority, Γ : 3, threshold: 0, k : 3
Class Noise cleaning with noise scoring	CNC-NOS	–	Base filters: {IPF, NCNEdit, RNG, CVCF, EF}, k : 5

por cada dataset y nivel de ruido.

5.1.2 Técnicas de filtrado a evaluar

Las técnicas de filtrado a evaluar son las mostradas en la tabla 5.2, donde además se especifican los hiperparámetros con los que configurarlas.

5.1.3 Metodología de evaluación

De modo que para cada uno de los 5 entrenamientos de las 5 validaciones cruzadas a realizar para cada dataset, nivel de ruido y semilla, se comenzará filtrando los datos mediante la técnica que corresponda de las mostradas en la Tabla 5.2 para posteriormente usar dichos datos para, por separado, entrenar los clasificadores de la Tabla 5.3, con la configuración dada por los hiperparámetros adjuntos en la misma.

Table 5.3: Parameter setting for the classifiers.

Classifier	Parameters
C4.5	Pruning: yes, Confidence: 0.25, Instances per leaf: 2
k -NN	k : 1, Distance: Euclidean/HVDM
SVM	C: 100, Kernel: RBF, ϵ : 1E-12, γ : 0.3

Esperamos con ello obtener los mismos (si no muy similares) resultados mostrados en [16].

5.2 Evaluación de la propuesta

5.2.1 Conjuntos de datos utilizados

De cara a evaluar el desempeño de los filtros basados en el empleo de TabPFN como clasificador base se hará uso de los 30 conjuntos de datos mostrados en la tabla 5.4, disponibles en [este enlace](#).

Siguiendo la tabla 5.4 (con la misma estructura que la tabla 5.1), haremos uso de 30 conjuntos de datos, empleando 3 niveles de ruido: 5% (bajo), 25% (medio) y 50%(alto), ejecutando 5 validaciones cruzadas estratificadas con solo la primera de las semillas disponibles en el enlace anterior (es decir seed=1).

5.2.2 Metodología de simulación

Dado un dataset y un nivel de ruido, la metodología a seguir en cada uno de los 5 entrenamientos a realizar en cada 5 validación cruzada constará de los siguientes pasos:

1. Preprocesar los datos mediante una imputación simple de valores perdidos (es decir, eliminando las instancias con algún valor faltante), seguida de una codificación tipo one-hot-encoding de los atributos nominales y una estandarización de los datos.
2. Aplicación de la técnica de filtrado que corresponda, con la que obtener un conjunto filtrado de instancias potencialmente ruidosas.
3. Uso del conjunto anteriormente filtrado para entrenar una regresión logística (por ser computacionalmente menos costoso que alternativas como RandomForest).
4. Validación del modelo anteriormente obtenido sobre el fold de testeo restante, de donde extraer las métricas de clasificación y limpieza a analizar.

Para finalmente promediar los resultados obtenidos en el paso 4 sobre cada 5 validación cruzada.

Table 5.4: Datasets a emplear para la evaluación de los filtros basados en TabPFN.

Dataset	#EX	#AT	#CL	Dataset	#EX	#AT	#CL
zoo	101	16(0/16)	7	monk	432	6(6/0)	2
lymphography	148	18(3/15)	4	led7digit	500	7(0/7)	10
iris	150	4(4/0)	3	page-blocks	548	10(10/0)	5
automobile	159	25(15/10)	6	wdbc	569	30(30/0)	2
hayes-roth	160	4(4/0)	3	balance	625	4(4/0)	3
wine	178	13(13/0)	3	satimage	643	36(36/0)	7
sonar	208	60(60/0)	2	thyroid	720	21(6/15)	3
glass	214	9(9/0)	7	pima	768	8(8/0)	2
new-thyroid	215	5(5/0)	3	vehicle	846	18(18/0)	4
heart	270	13(13/0)	2	vowel	990	13(13/0)	11
cleveland	297	13(13/0)	5	german	1000	20(13/7)	2
splice	319	60(0/60)	3	flare	1066	11(0/11)	6
ecoli	336	7(7/0)	8	penbased	1099	16(16/0)	10
ionosphere	351	33(33/0)	2	nursery	1296	8(0/8)	5
dermatology	358	33(1/32)	6	contraceptive	1473	9(9/0)	3

Esto se hará para todos y cada uno de los conjuntos de datos disponibles, con los niveles y tipos de ruido antes propuestos.

5.2.3 Métricas de evaluación

Para comparar el desempeño de todos y cada uno de los filtros pondremos nuestra atención en:

- La diferencia absoluta entre el accuracy balanceado tras filtrar y sin filtrar los datos de entrada ($\xi_{BalAcc} = BalAcc_{filtrado} - BalAcc_{base}$). Al considerar el accuracy balanceado, en lugar del básico, evitaremos encubrir cualquier sesgo de las técnicas a aplicar hacia las clases mayoritarias, si las hubiera, algo a tener en cuenta siempre que se apliquen técnicas que añadan y eliminen instancias. Optamos por tomar la diferencia (en lugar del accuracy balanceado original) porque la creemos más representativa del nivel de mejora que podemos esperar al aplicar cada filtro, independientemente de si el accuracy del que se parte antes de filtrar es mayor o menor. Adicionalmente se consideró analizar también métricas de clasificación como la precisión y la puntuación f_1 , pero tras hacerlo se observó siempre una correlación casi perfecta entre tales métricas, decidiendo con ello excluirlas del análisis con el objetivo de hacer más sencilla y limpia la comprensión de los resultados.
- Porcentaje de instancias etiquetadas como ruidosas, junto con *accuracy*, *precision*, *recall*, f_1 , *specificity* y *MCC* para evaluar la capacidad del filtro de identificar correctamente las instancias a las que se les ha añadido ruido.
- Tiempo de filtrado, para valorar la eficiencia computacional de las técnicas empleadas.

Con ello trataremos de medir no solo las mejoras en el poder de clasificación de los modelos a emplear tras el filtrado, sino también el poder de detección de ruido de los filtros previamente empleados.

5.2.4 Configuración de los filtros basados en distancias

Por un lado compararemos el desempeño de la propuesta con los filtros basados en distancias de la tabla 5.5. Al contrario de como se realiza en la sección anterior, en esta, con el objetivo de obtener resultados lo más fieles a la realidad posibles, para cada par (dataset, nivel de ruido) se lanzará cada filtro con diferentes configuraciones de hiperparámetros, procurando así considerar solo aquella que mejores resultados reporte para cada uno de los pares anteriores.

De tal modo se ejecutará cada técnica de filtrado con, donde corresponda, todos y cada uno de los hiperparámetros adjuntos en la tabla 5.6.

Así se probará, por ejemplo, *ENN* con 3,5 y 9 vecinos; o *Multiedit* con 3, 5 o 9 vecinos, así como 3 o 5 particiones y un máximo de 3 iteraciones del algoritmo.

Abreviatura	Filtro
ENN	Edited Nearest Neighbors
ENNProb	Edited Nearest Neighbors with Probabilities
ENNProbTh	Edited Nearest Neighbors with Probabilities and Threshold
NCNE	Nearest Centroid Neighbor Edition
Multiedit	Multiedit

Table 5.5

Hiperparámetro	Símbolo	Valores considerados
Tipo de voto	k	{3,5,9}
Umbral de decisión	τ	{0.5,0.7}
Número de bloques / particiones	b	{3,5}
Máximo número de iteraciones / repeticiones	it	{3}

Table 5.6: Hiperparámetros considerados en los filtros basados en distancia.

5.2.5 Configuración de los filtros basados en clasificadores

Por otro lado se comparará el desempeño de la propuesta frente a los filtros basados en clasificadores de la tabla 5.7 haciendo uso, de manera análoga al apartado anterior, de los hiperparámetros de la tabla 5.8.

Abreviatura	Filtro
CF	Classification Filter
CVCF	Cross-Validated Committees Filter
CVCFTh	Cross-Validated Committees Filter with Threshold
EF	Ensemble Filtering
EFTh	Ensemble Filtering with Threshold
IPF	Iterative Partitioning Filter
INFFC	Iterative Noise Filter based on the Fusion of Classifiers
INFFCTh	Iterative Noise Filter based on the Fusion of Classifiers with Threshold

Table 5.7: Filtros basados en clasificadores considerados.

Donde, por ejemplo, ejecutaremos CF, EF/EFTh e INFFC/INFFCTh haciendo uso de los clasificadores base de la tabla 5.9 (empleándolos individualmente en el CF y en el voto mediante ensemble en los demás). Para el resto de filtros (CVCF/CVCFTh e IPF/IPFTh) se ha empleado solo el clasificador tipo C4.5.

5.2.6 Configuración de las técnicas de limpieza de clase

Para el filtro CNCNOS se consideró una rejilla específica de hiperparámetros siguiendo la tabla 5.10.

5.2.7 Metodología de evaluación de los resultados

Comenzaremos evaluando la propuesta atendiendo al porcentaje de mejora promedio (sobre todos los conjuntos de datos) en el accuracy balanceado obtenido con cada técnica de

Hiperparámetro	Símbolo	Valores considerados
Número de folds de validación cruzada	cv	$\{5, 7, 10\}$
Umbral de decisión	τ	$\{0.5, 0.7\}$
Número de particiones	p	$\{5, 7, 10\}$
Regla de decisión / votación	r	{consensus, majority, threshold}
Máximo número de iteraciones	it	$\{3\}$
Umbral de parada	p_{stop}	$\{0.03\}$
Paciencia de parada	k	$\{3\}$
Fracción máxima de instancias eliminadas	f_{max}	$\{0.5\}$

Table 5.8: Hiperparámetros considerados en los filtros basados en clasificadores.

Abreviatura	Clasificador base
DT	Decision Tree con criterio de entropía
C45	Árbol de decisión tipo C4.5
KNN ₁	k -Nearest Neighbors con $k = 3$
LR	Logistic Regression

Table 5.9: Clasificadores base utilizados en los filtros basados en clasificadores.

Hiperparámetro	Símbolo	Valores considerados
Número de vecinos (base y para puntuación)	k_b	$\{3, 5, 9\}$
Número de folds internos	cv	$\{5, 7, 9\}$
Máximo número de iteraciones	it	$\{3\}$

Table 5.10: Hiperparámetros considerados para el filtro CNCNOS.

filtrado, según el nivel de ruido introducido. Para reforzar los resultados, dicha estimación irá acompañada del p-valor asociado al test de wilcoxon de doble sentido ejecutado para cada filtro sobre las diferencias obtenidas con todos los conjuntos de datos usados.

Adicionalmente acompañaremos este punto con un estudio de la variabilidad de los filtros empleados haciendo uso de diagramas de cajas sobre las diferencias obtenidas con cada filtro para cada nivel de ruido.

Así mismo visualizaremos el porcentaje de conjuntos de datos en los que cada filtro a reportado una mejora (es decir, en los que la diferencia promedio por dataset era positiva).

Este análisis será realizado a dos niveles:

- **A nivel de familia**, donde cada técnica de filtrado será considerada como una familia de métodos parametrizada. En este caso, para cada conjunto de datos y partición de la validación cruzada (5CV), se empleará la configuración que reporte la mayor mejora en el *accuracy* balanceado entre todas las hiperconfiguraciones evaluadas para dicha familia. Es decir, se compararán, por ejemplo, el desempeño de ENN frente a CF, ello a partir de los mejores resultados obtenidos con cada uno de los anteriores filtros para cada dataset, independientemente de que la hiperparametrización sea diferente entre estos. De este modo, el objetivo es estimar el rendimiento potencial máximo alcanzable por cada propuesta, independientemente de la combinación concreta de hiperparámetros utilizada.
- **A nivel de hiperparametrización**, donde cada combinación específica de hiperparámetros será analizada de forma independiente. Este estudio permitirá evaluar la robustez y estabilidad de las distintas configuraciones, así como determinar si las mejoras observadas a nivel de familia se mantienen de manera consistente o dependen de una selección concreta de hiperparámetros.

En tales líneas planteamos estudiar las siguientes hipótesis 1

- **H₁**: La propuesta basada en *TabPFN* constituye, a nivel de familia de técnicas, el método de filtrado que produce una mayor mejora en el *accuracy* balanceado respecto al resto de alternativas consideradas.
- **H₂**: Existe al menos una hiperconfiguración de la propuesta basada en *TabPFN* cuyo desempeño es estadísticamente superior al de las mejores hiperconfiguraciones de los restantes filtros evaluados.
- **H₃**: Las mejoras obtenidas por la propuesta son estadísticamente significativas respecto a la ausencia de filtrado, según el contraste de Wilcoxon sobre las diferencias obtenidas en los distintos conjuntos de datos.
- **H₄**: La propuesta presenta una mayor robustez que los métodos comparados, entendiendo como tal una menor variabilidad en las mejoras obtenidas entre diferentes conjuntos de datos y escenarios de ruido.
- **H₅**: La superioridad de la propuesta se mantiene para distintos niveles de ruido, no limitándose únicamente a escenarios con porcentajes elevados de instancias ruidosas.
- **H₆**: La propuesta consigue mejorar el rendimiento predictivo en un mayor porcentaje de conjuntos de datos que el resto de técnicas de filtrado consideradas.

- **H₇**: La influencia de la hiperparametrización sobre el rendimiento de la propuesta es menor que la observada en las técnicas competidoras, indicando una mayor estabilidad frente a la elección de parámetros.

Posteriormente compararemos cara a cara la técnica aquí propuesta frente todos y cada uno de los filtros empleados. Primero realizaremos una comparación por pares en la que estudiar las diferencias entre las mejoras obtenidas con el par correspondiente a la propuesta junto con cada uno de los demás filtros; y segundo, como medida global computaremos el ranking promedio de cada filtro (sobre los conjuntos de datos empleados).

Tras analizar el desempeño en clasificación de la propuesta, realizaremos un análisis muy similar, atendiendo sin embargo esta vez a la capacidad para detectar instancias potencialmente ruidosas.

A continuación nos centraremos en el costo computacional de la misma frente a las demás, ello representando el tiempo promedio empleado en el ciclo entrenamiento-validación en cada conjuntos de datos, en función del nivel de ruido añadido. Comprobaremos así si se observa alguna relación entre dicho nivel de ruido y el tiempo de filtrado de los datos.

5.2.8 Especificaciones del ordenador utilizado para realizar las simulaciones

Todos los cálculos y simulaciones presentados en este trabajo se realizaron en un ordenador portátil *ASUS ROG Zephyrus G14 GA401QC*. El equipo cuenta con un procesador AMD Ryzen 7 5800HS, formado por 8 núcleos y 16 hilos de ejecución, con una frecuencia máxima de hasta 4.4 GHz. Además, dispone de una tarjeta gráfica NVIDIA GeForce RTX 3050 Laptop GPU con 4 GB de memoria GDDR6, aunque las simulaciones se ejecutaron principalmente sobre CPU. La memoria principal del sistema es de 24 GB de RAM DDR4. Todo se realizó sobre el kernel de Linux en la distribución de Arch+Hyprland.

6 Análisis de los resultados

6.1 Evaluación de la biblioteca

6.2 Evaluación de la propuesta

6.2.1 Desempeño global promedio

A nivel de familia

Comenzamos mostrando en el margen izquierdo de la tabla 6.1 la mejora promedio en el accuracy balanceado (ζ_{BalAcc}) obtenida con cada técnica de filtrado sobre los conjuntos de datos de la tabla 5.4, para cada nivel de ruido de los 3 propuestos. En el margen derecho de la misma tabla se adjunta el p-valor asociado a dicha media. En negrita quedan resaltadas las más altas mejoras obtenidas para cada nivel de ruido, siempre que el p-valor asociado sea estadísticamente significativo (inferior a 0.05), lo que también se resalta en negrita en la correspondiente región.

Table 6.1: Mejoras promedio obtenidas al aplicar cada filtro para cada nivel de ruido, junto con el p-valor asociado a dichos promedios.

Method	ζ_{BalAcc}			p-value		
	5%	25%	50%	5%	25%	50%
LogReg						
ENN	-1.16	1.65	3.27	6.26E-1	1.12E-1	3.28E-1
ENNProb	-0.38	2.59	4.03	5.98E-1	1.45E-2	9.61E-2
ENNTh	-1.57	0.86	2.78	9.35E-1	1.98E-1	3.39E-1
MultiEdit	-14.11	-11.04	-10.11	3.79E-6	1.10E-4	5.51E-4
NCNEdit	-0.72	1.64	4.41	9.35E-1	1.77E-1	1.13E-2
CF	2.51	4.54	5.20	3.45E-5	1.82E-5	1.53E-4
CVCF	0.08	0.22	0.27	8.10E-1	7.21E-1	3.99E-2
CVCFTTh	-0.08	0.03	-0.12	7.49E-1	7.89E-1	3.26E-1
Ensemble	1.54	3.87	4.10	8.14E-3	1.05E-3	1.70E-4
EnsembleTh	1.84	3.97	4.73	2.77E-3	9.52E-4	1.86E-3
INFFC	0.46	3.14	5.22	1.95E-1	5.38E-3	2.56E-3
INFFCTTh	0.82	2.74	5.38	6.99E-2	1.28E-2	4.60E-4
IPF	1.21	2.49	3.71	3.15E-4	8.86E-5	2.60E-4
CNCNOS	-2.12	0.17	2.77	4.79E-1	7.62E-1	3.05E-1
TABPFN-CF	1.87	4.39	4.29	8.71E-3	4.03E-3	7.32E-2
TABPFN-CVCF	1.56	4.11	3.82	2.03E-3	2.62E-3	5.78E-2
TABPFN-CVCFTTh	2.02	4.76	4.66	2.03E-3	2.86E-4	1.74E-2
TABPFN-IPF	1.81	2.79	3.50	4.78E-5	4.93E-4	2.96E-3

Encontramos así que...

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam in lacus a nisi tincidunt congue. Vivamus sed sapien vitae augue aliquam auctor eget vel turpis. Aliquam erat vo-

lutupat. Aliquam turpis sapien, dignissim quis nisi eu, sollicitudin ultrices nibh. Vivamus at feugiat dolor, ut rhoncus sapien. Sed ut volutpat dolor. Vestibulum quis nulla imperdiet, semper libero ut, sodales dolor. Suspendisse sollicitudin augue vitae leo ultricies, quis condimentum justo elementum. Cras sed sem suscipit nisi volutpat fringilla. Nam eleifend nibh quis nisl molestie, sed pharetra tellus tempus. Integer facilisis fringilla nisi id efficitur. Nullam nec tincidunt nisi, a lobortis justo. Nunc vitae erat urna. Sed urna dui, tincidunt a velit sit amet, facilisis placerat magna. Curabitur viverra augue et cursus vehicula. Sed ornare imperdiet est at eleifend. In eu sagittis diam. Etiam feugiat turpis quis est elementum, id luctus ligula ornare. Etiam sapien odio, commodo eget tempor quis, maximus et purus. Donec risus ex, gravida eu faucibus quis, ornare sit amet libero. Sed ut cursus tortor, eget porta nibh. Integer elementum sit amet diam id eleifend. Duis a ante eu turpis eleifend ultricies. Pellentesque malesuada in sapien at aliquet. Maecenas viverra scelerisque elit. Nulla tincidunt nisl nec lacus pellentesque, id interdum risus aliquam. Proin leo dui, malesuada at justo vitae, auctor pharetra sapien.

A continuación ponemos nuestra atención en la figura 6.1 donde mostramos los boxplots asociados a las diferencias en el accuracy balanceado obtenidas tras aplicar cada uno de los filtros antes mostrados.

Se aprecia con ello que...

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam in lacus a nisi tincidunt congue. Vivamus sed sapien vitae augue aliquam auctor eget vel turpis. Aliquam erat volutpat. Aliquam turpis sapien, dignissim quis nisi eu, sollicitudin ultrices nibh. Vivamus at feugiat dolor, ut rhoncus sapien. Sed ut volutpat dolor. Vestibulum quis nulla imperdiet, semper libero ut, sodales dolor. Suspendisse sollicitudin augue vitae leo ultricies, quis condimentum justo elementum. Cras sed sem suscipit nisi volutpat fringilla. Nam eleifend nibh quis nisl molestie, sed pharetra tellus tempus. Integer facilisis fringilla nisi id efficitur. Nullam nec tincidunt nisi, a lobortis justo. Nunc vitae erat urna. Sed urna dui, tincidunt a velit sit amet, facilisis placerat magna. Curabitur viverra augue et cursus vehicula. Sed ornare imperdiet est at eleifend. In eu sagittis diam. Etiam feugiat turpis quis est elementum, id luctus ligula ornare. Etiam sapien odio, commodo eget tempor quis, maximus et purus. Donec risus ex, gravida eu faucibus quis, ornare sit amet libero. Sed ut cursus tortor, eget porta nibh. Integer elementum sit amet diam id eleifend. Duis a ante eu turpis eleifend ultricies. Pellentesque malesuada in sapien at aliquet. Maecenas viverra scelerisque elit. Nulla tincidunt nisl nec lacus pellentesque, id interdum risus aliquam. Proin leo dui, malesuada at justo vitae, auctor pharetra sapien.

6 Análisis de los resultados

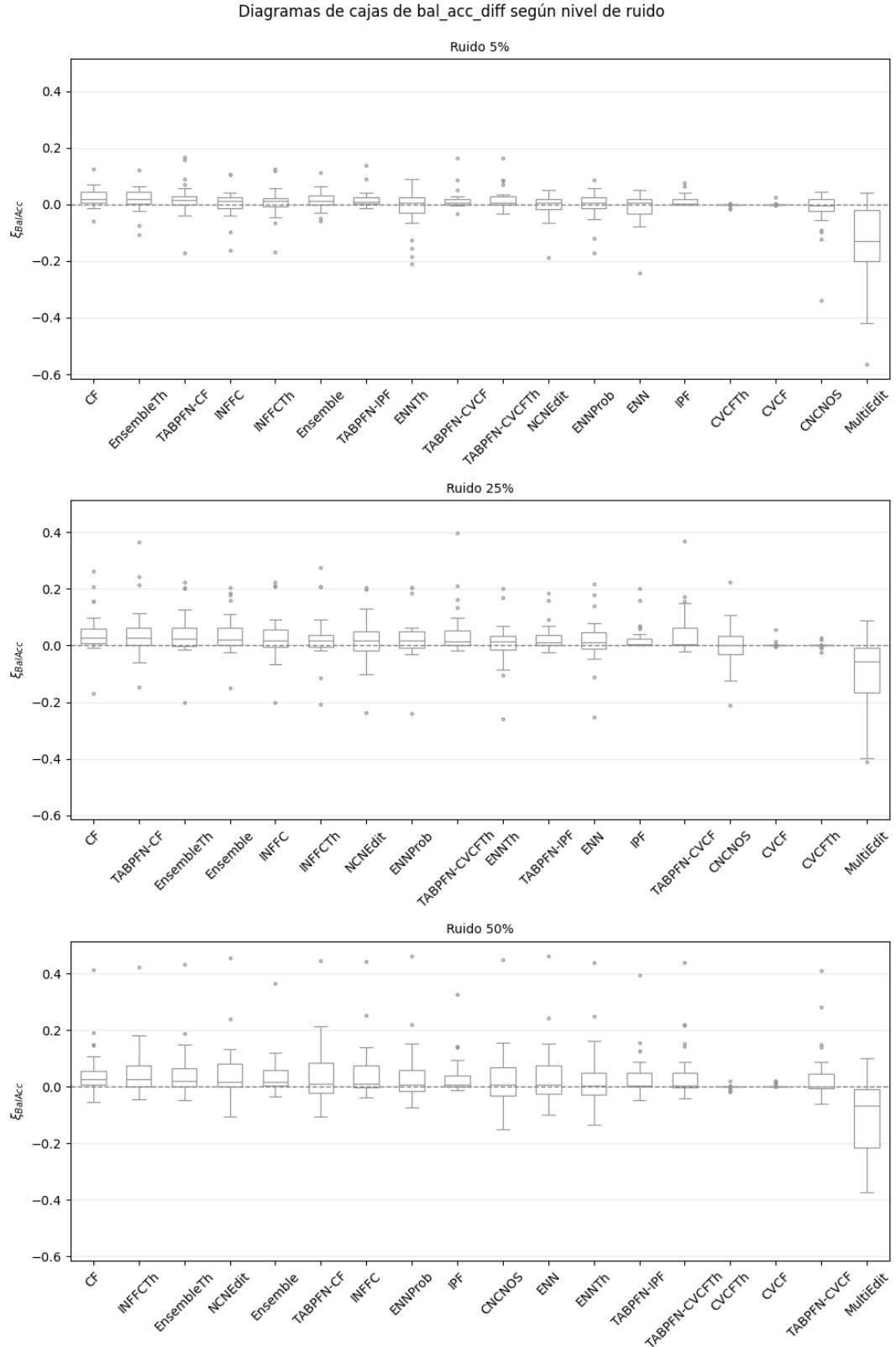


Figure 6.1: Diagramas de cajas correspondientes a las diferencias entre el accuracy balanceado tras y sin filtrar, según el nivel de ruido y el filtro aplicado.

Si antedemos a las medianas de las figuras anteriores descubriremos que en realidad la mejora promedio puede en algunos casos no ser suficientemente representativa del poder de cada filtro (dada la variabilidad de las diferencias promediadas sobre todos y cada uno de los datasets empleados). Por ello podemos fijar nuestra atención también en la figura 6.2, en la que se muestra el porcentaje de conjuntos de datos en los que cada filtro a logrado una mejora, junto con los p-valores asociados a que dicho porcentaje es mayor al 50%.

De lo anterior extraemos que...

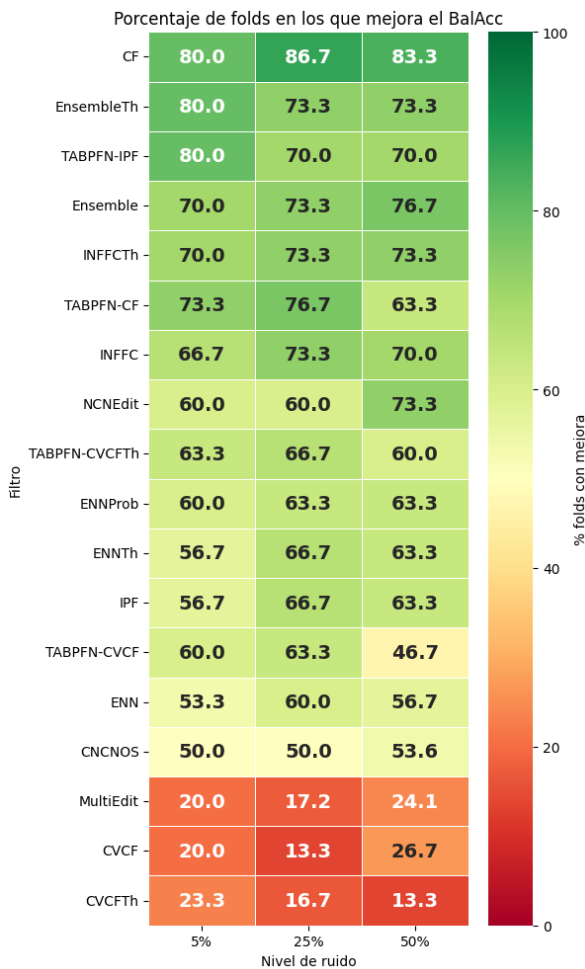


Figure 6.2: Porcentaje de folds en los que se ha obtenido una mejora en el accuracy balanceado según filtro y nivel de ruido.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam in lacus a nisi tincidunt congue. Vivamus sed sapien vitae augue aliquam auctor eget vel turpis. Aliquam erat volutpat. Aliquam turpis sapien, dignissim quis nisi eu, sollicitudin ultrices nibh. Vivamus at feugiat dolor, ut rhoncus sapien. Sed ut volutpat dolor. Vestibulum quis nulla imperdiet, semper libero ut, sodales dolor. Suspendisse sollicitudin augue vitae leo ultricies, quis condimentum justo elementum. Cras sed sem suscipit nisi volutpat fringilla. Nam eleifend nibh quis nisl molestie, sed pharetra tellus tempus. Integer facilisis fringilla nisi id efficitur. Nullam nec tincidunt nisi, a lobortis justo. Nunc vitae erat urna. Sed urna dui, tincidunt a velit sit amet, facilisis placerat magna. Curabitur viverra augue et cursus vehicula.

Sed ornare imperdiet est at eleifend. In eu sagittis diam. Etiam feugiat turpis quis est elementum, id luctus ligula ornare. Etiam sapien odio, commodo eget tempor quis, maximus et purus. Donec risus ex, gravida eu faucibus quis, ornare sit amet libero. Sed ut cursus tortor, eget porta nibh. Integer elementum sit amet diam id eleifend. Duis a ante eu turpis eleifend ultricies. Pellentesque malesuada in sapien at aliquet. Maecenas viverra scelerisque elit. Nulla tincidunt nisl nec lacus pellentesque, id interdum risus aliquam. Proin leo dui, malesuada at justo vitae, auctor pharetra sapien.

A nivel de hiperparametrización

En este caso, al contar con 85 filtros diferentes (considerando las diferencias existente entre una misma familia según la hiperparametrización realizada), en las tablas 6.2 mostramos

el top 20 de modelos, indicada su hiperparametrización en la primera columna, ordenados por la mejora promedio obtenida (en la segunda columna, ξ_{BalAcc}), junto al p-valor asociado al test de Wilcoxon hecho sobre las diferencias obtenidas en cada dataset, ello para los 3 niveles de ruido simulados.

De lo anterior extraemos que...

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam in lacus a nisi tincidunt congue. Vivamus sed sapien vitae augue aliquam auctor eget vel turpis. Aliquam erat volutpat. Aliquam turpis sapien, dignissim quis nisi eu, sollicitudin ultrices nibh. Vivamus at feugiat dolor, ut rhoncus sapien. Sed ut volutpat dolor. Vestibulum quis nulla imperdiet, semper libero ut, sodales dolor. Suspendisse sollicitudin augue vitae leo ultricies, quis condimentum justo elementum. Cras sed sem suscipit nisi volutpat fringilla. Nam eleifend nibh quis nisl molestie, sed pharetra tellus tempus. Integer facilisis fringilla nisi id efficitur. Nullam nec tincidunt nisi, a lobortis justo. Nunc vitae erat urna. Sed urna dui, tincidunt a velit sit amet, facilisis placerat magna. Curabitur viverra augue et cursus vehicula. Sed ornare imperdiet est at eleifend. In eu sagittis diam. Etiam feugiat turpis quis est elementum, id luctus ligula ornare. Etiam sapien odio, commodo eget tempor quis, maximus et purus. Donec risus ex, gravida eu faucibus quis, ornare sit amet libero. Sed ut cursus tortor, eget porta nibh. Integer elementum sit amet diam id eleifend. Duis a ante eu turpis eleifend ultricies. Pellentesque malesuada in sapien at aliquet. Maecenas viverra scelerisque elit. Nulla tincidunt nisl nec lacus pellentesque, id interdum risus aliquam. Proin leo dui, malesuada at justo vitae, auctor pharetra sapien.

6.2.2 Desempeño frente a las demás técnicas

A nivel de familia

En la tabla 6.3 se muestran la media, mediana y el p-valor sobre las diferencias entre la mejora obtenida con el filtro propuesto y todos los demás del conjunto.

De ello deducimos que...

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam in lacus a nisi tincidunt congue. Vivamus sed sapien vitae augue aliquam auctor eget vel turpis. Aliquam erat volutpat. Aliquam turpis sapien, dignissim quis nisi eu, sollicitudin ultrices nibh. Vivamus at feugiat dolor, ut rhoncus sapien. Sed ut volutpat dolor. Vestibulum quis nulla imperdiet, semper libero ut, sodales dolor. Suspendisse sollicitudin augue vitae leo ultricies, quis condimentum justo elementum. Cras sed sem suscipit nisi volutpat fringilla. Nam eleifend nibh quis nisl molestie, sed pharetra tellus tempus. Integer facilisis fringilla nisi id efficitur. Nullam nec tincidunt nisi, a lobortis justo. Nunc vitae erat urna. Sed urna dui, tincidunt a velit sit amet, facilisis placerat magna. Curabitur viverra augue et cursus vehicula. Sed ornare imperdiet est at eleifend. In eu sagittis diam. Etiam feugiat turpis quis est elementum, id luctus ligula ornare. Etiam sapien odio, commodo eget tempor quis, maximus et purus. Donec risus ex, gravida eu faucibus quis, ornare sit amet libero. Sed ut cursus tortor, eget porta nibh. Integer elementum sit amet diam id eleifend. Duis a ante eu turpis eleifend ultricies. Pellentesque malesuada in sapien at aliquet. Maecenas viverra scelerisque elit. Nulla tincidunt nisl nec lacus pellentesque, id interdum risus aliquam. Proin leo dui,

malesuada at justo vitae, auctor pharetra sapien.

Por otro lado, en la figura 6.3 mostramos el ranking promedio de cada filtro en la evaluación de estos sobre cada uno de los datasets, junto con el p-valor asociado a cada uno de los mismos.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam in lacus a nisi tincidunt congue. Vivamus sed sapien vitae augue aliquam auctor eget vel turpis. Aliquam erat volutpat. Aliquam turpis sapien, dignissim quis nisi eu, sollicitudin ultrices nibh. Vivamus at feugiat dolor, ut rhoncus sapien. Sed ut volutpat dolor. Vestibulum quis nulla imperdiet, semper libero ut, sodales dolor. Suspendisse sollicitudin augue vitae leo ultricies, quis condimentum justo elementum. Cras sed sem suscipit nisi volutpat fringilla. Nam eleifend nibh quis nisl molestie, sed pharetra tellus tempus. Integer facilisis fringilla nisi id efficitur. Nullam nec tincidunt nisi, a lobortis justo. Nunc vitae erat urna. Sed urna dui, tincidunt a velit sit amet, facilisis placerat magna. Curabitur viverra augue et cursus vehicula.

Sed ornare imperdiet est at eleifend. In eu sagittis diam. Etiam feugiat turpis quis est elementum, id luctus ligula ornare. Etiam sapien odio, commodo eget tempor quis, maximus et purus. Donec risus ex, gravida eu faucibus quis, ornare sit amet libero. Sed ut cursus tortor, eget porta nibh. Integer elementum sit amet diam id eleifend. Duis a ante eu turpis eleifend ultricies. Pellentesque malesuada in sapien at aliquet. Maecenas viverra scelerisque elit. Nulla tincidunt nisl nec lacus pellentesque, id interdum risus aliquam. Proin leo dui, malesuada at justo vitae, auctor pharetra sapien.

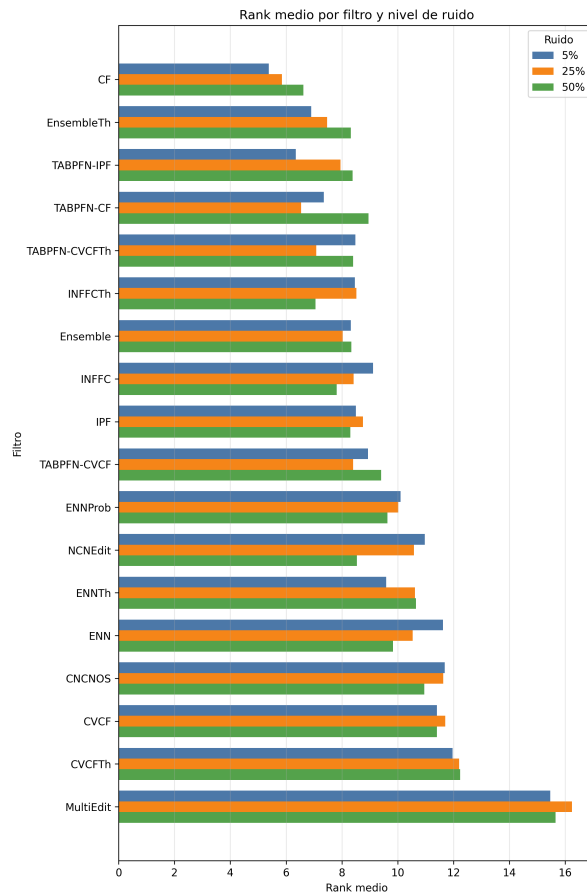


Figure 6.3: Ranking promedio de cada filtro

A nivel de hiperparametrización

Siguiendo los resultados obtenidos en la sección ??, si seleccionáramos ¿?

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam in lacus a nisi tincidunt congue. Vivamus sed sapien vitae augue aliquam auctor eget vel turpis. Aliquam erat volutpat. Aliquam turpis sapien, dignissim quis nisi eu, sollicitudin ultrices nibh. Vivamus at feugiat dolor, ut rhoncus sapien. Sed ut volutpat dolor. Vestibulum quis nulla imperdiet,

semper libero ut, sodales dolor. Suspendisse sollicitudin augue vitae leo ultricies, quis condimentum justo elementum. Cras sed sem suscipit nisi volutpat fringilla. Nam eleifend nibh quis nisl molestie, sed pharetra tellus tempus. Integer facilisis fringilla nisi id efficitur. Nullam nec tincidunt nisi, a lobortis justo. Nunc vitae erat urna. Sed urna dui, tincidunt a velit sit amet, facilisis placerat magna. Curabitur viverra augue et cursus vehicula. Sed ornare imperdiet est at eleifend. In eu sagittis diam. Etiam feugiat turpis quis est elementum, id luctus ligula ornare. Etiam sapien odio, commodo eget tempor quis, maximus et purus. Donec risus ex, gravida eu faucibus quis, ornare sit amet libero. Sed ut cursus tortor, eget porta nibh. Integer elementum sit amet diam id eleifend. Duis a ante eu turpis eleifend ultricies. Pellentesque malesuada in sapien at aliquet. Maecenas viverra scelerisque elit. Nulla tincidunt nisl nec lacus pellentesque, id interdum risus aliquam. Proin leo dui, malesuada at justo vitae, auctor pharetra sapien.

Por otro lado, atendiendo a la figura 6.4 encontramos el ranking promedio por modelo específico:

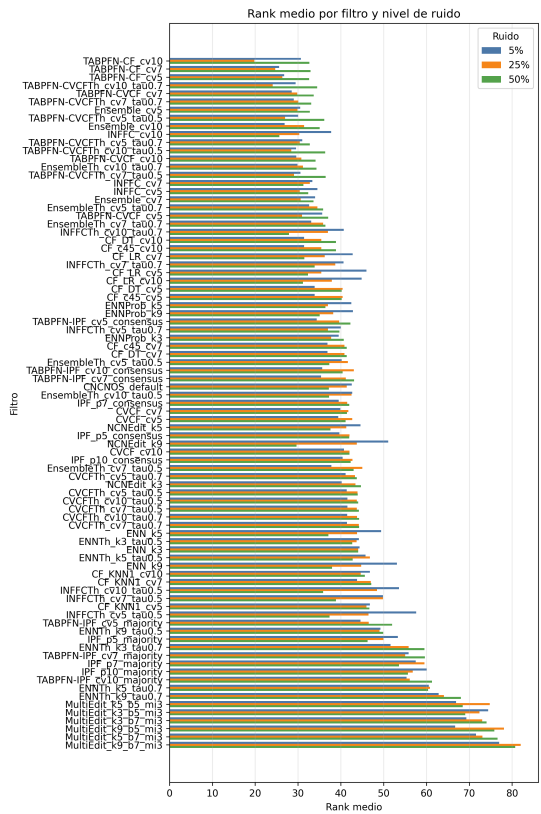


Figure 6.4: Caption

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam in lacus a nisi tincidunt congue. Vivamus sed sapien vitae augue aliquam auctor eget vel turpis. Aliquam erat volutpat. Aliquam turpis sapien, dignissim quis nisi eu, sollicitudin ultrices nibh. Vivamus at feugiat dolor, ut rhoncus sapien. Sed ut volutpat dolor. Vestibulum quis nulla imperdiet,

semper libero ut, sodales dolor. Suspendisse sollicitudin augue vitae leo ultricies, quis condimentum justo elementum. Cras sed sem suscipit nisi volutpat fringilla. Nam eleifend nibh quis nisl molestie, sed pharetra tellus tempus. Integer facilisis fringilla nisi id efficitur. Nullam nec tincidunt nisi, a lobortis justo. Nunc vitae erat urna. Sed urna dui, tincidunt a velit sit amet, facilisis placerat magna. Curabitur viverra augue et cursus vehicula.

Sed ornare imperdiet est at eleifend. In eu sagittis diam. Etiam feugiat turpis quis est elementum, id luctus ligula ornare. Etiam sapien odio, commodo eget tempor quis, maximus et purus. Donec risus ex, gravida eu faucibus quis, ornare sit amet libero. Sed ut cursus tortor, eget porta nibh. Integer elementum sit amet diam id eleifend. Duis a ante eu turpis eleifend ultricies. Pellentesque malesuada in sapien at aliquet. Maecenas viverra scelerisque elit. Nulla tincidunt nisl nec lacus pellentesque, id interdum risus aliquam. Proin leo dui, malesuada at justo vitae, auctor pharetra sapien.

6.2.3 Análisis del tiempo de ejecución

En las figuras ?? se muestra un mapa de calor con el tiempo promedio sobre las 5CV en segundos que cada técnica ha tomado en filtrar los datos de entrenamiento:

Observamos así que...

Por otro lado podríamos poner nuestra atención a cómo escalan dichos tiempo en función del número de instancias a procesar. Si nos fijamos en la figura ??

En las mismas líneas podríamos cuestionarnos si el número de atributos afecta en algún modo al mismo tiempo de cómputo, lo que podemos analizar...

6.2.4 Capacidad de detección de instancias ruidosas

Una vez analizadas la potencial mejora en la etapa de clasificación así como el costo computacional de la propuesta, ambas frente a los demás modelos, pasamos a analizar la capacidad de detección de instancias potencialmente ruidosas.

A nivel de familia

Para ello comenzaremos poniendo nuestra atención en la tabla ??

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam in lacus a nisi tincidunt congue. Vivamus sed sapien vitae augue aliquam auctor eget vel turpis. Aliquam erat volutpat. Aliquam turpis sapien, dignissim quis nisi eu, sollicitudin ultrices nibh. Vivamus at feugiat dolor, ut rhoncus sapien. Sed ut volutpat dolor. Vestibulum quis nulla imperdiet, semper libero ut, sodales dolor. Suspendisse sollicitudin augue vitae leo ultricies, quis condimentum justo elementum. Cras sed sem suscipit nisi volutpat fringilla. Nam eleifend nibh quis nisl molestie, sed pharetra tellus tempus. Integer facilisis fringilla nisi id efficitur. Nullam nec tincidunt nisi, a lobortis justo. Nunc vitae erat urna. Sed urna dui, tincidunt a velit sit amet, facilisis placerat magna. Curabitur viverra augue et cursus vehicula. Sed ornare imperdiet est at eleifend. In eu sagittis diam. Etiam feugiat turpis quis est elementum, id luctus ligula ornare. Etiam sapien odio, commodo eget tempor quis, maximus et purus. Donec risus ex, gravida eu faucibus quis, ornare sit amet libero. Sed ut cursus tortor, eget porta nibh. Integer elementum sit amet diam id eleifend. Duis a ante eu turpis eleifend ultricies. Pellentesque malesuada in sapien at aliquet. Maecenas viverra scelerisque elit. Nulla tincidunt nisl nec lacus pellentesque, id interdum risus aliquam. Proin leo dui, malesuada at justo vitae, auctor pharetra sapien.

Si nos fijamos en los boxplots asociados a cada familia:

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam in lacus a nisi tincidunt congue. Vivamus sed sapien vitae augue aliquam auctor eget vel turpis. Aliquam erat volutpat. Aliquam turpis sapien, dignissim quis nisi eu, sollicitudin ultrices nibh. Vivamus at feugiat dolor, ut rhoncus sapien. Sed ut volutpat dolor. Vestibulum quis nulla imperdiet, semper libero ut, sodales dolor. Suspendisse sollicitudin augue vitae leo ultricies, quis condimentum justo elementum. Cras sed sem suscipit nisi volutpat fringilla. Nam eleifend nibh

Figure 6.5: Desempeño promedio en marcado de instancias ruidosas según la familia de filtros y el nivel de ruido inducido.

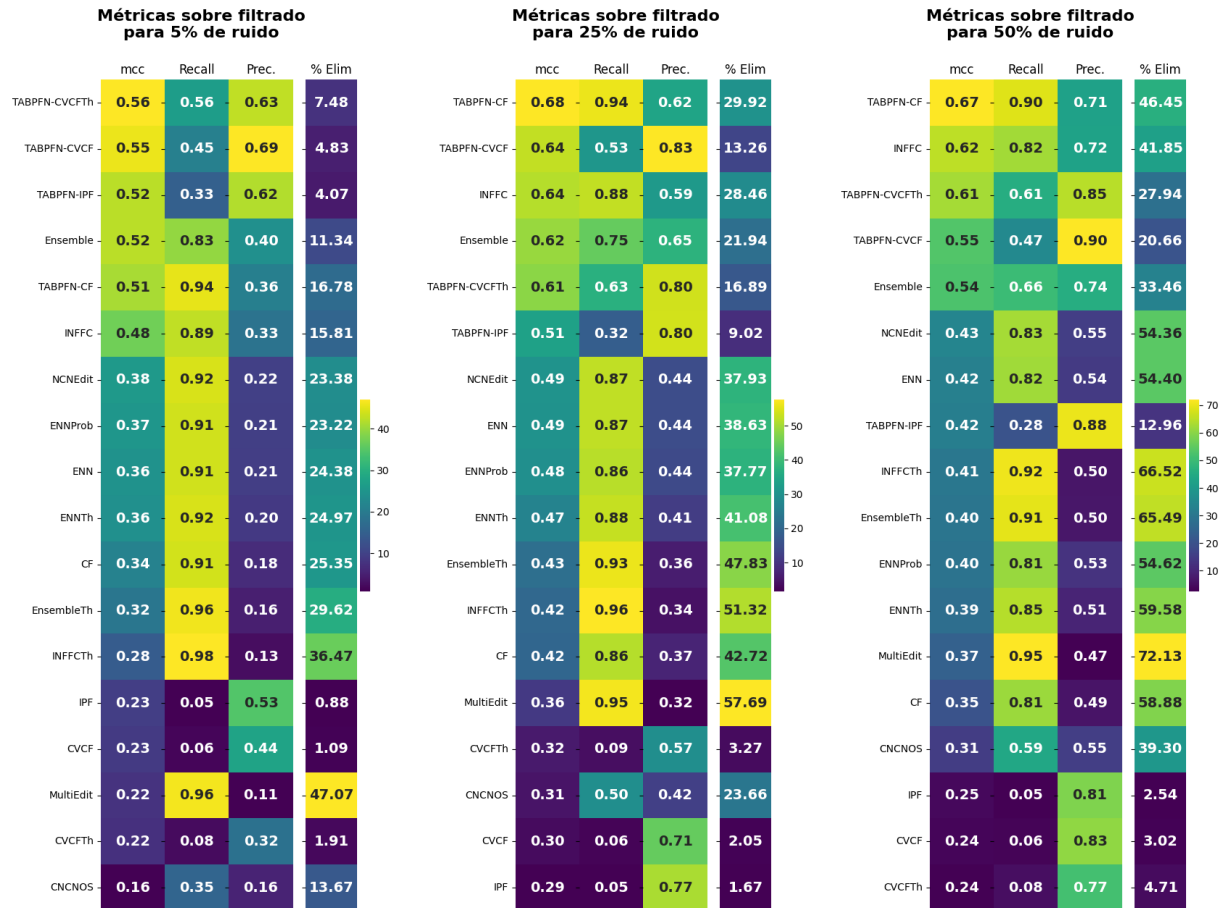
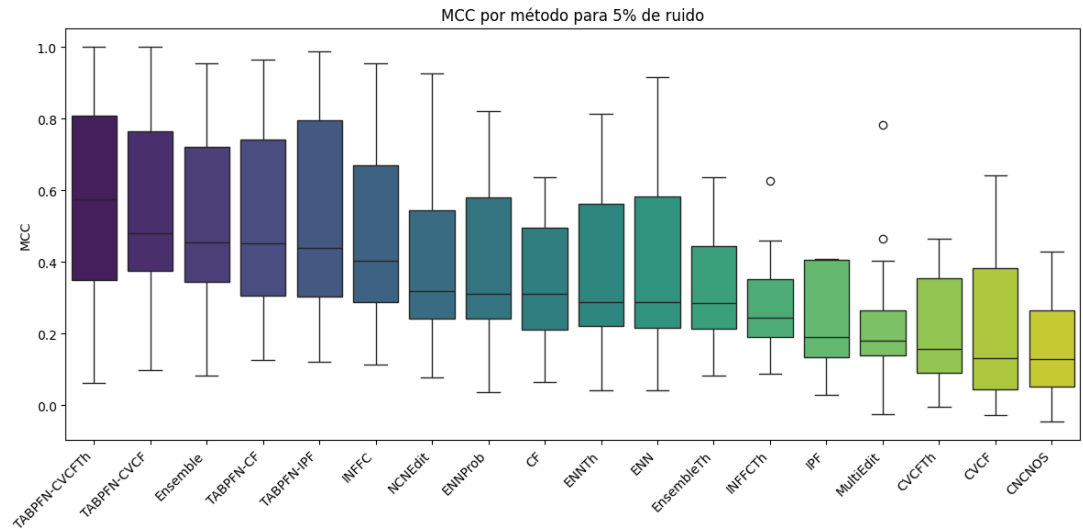
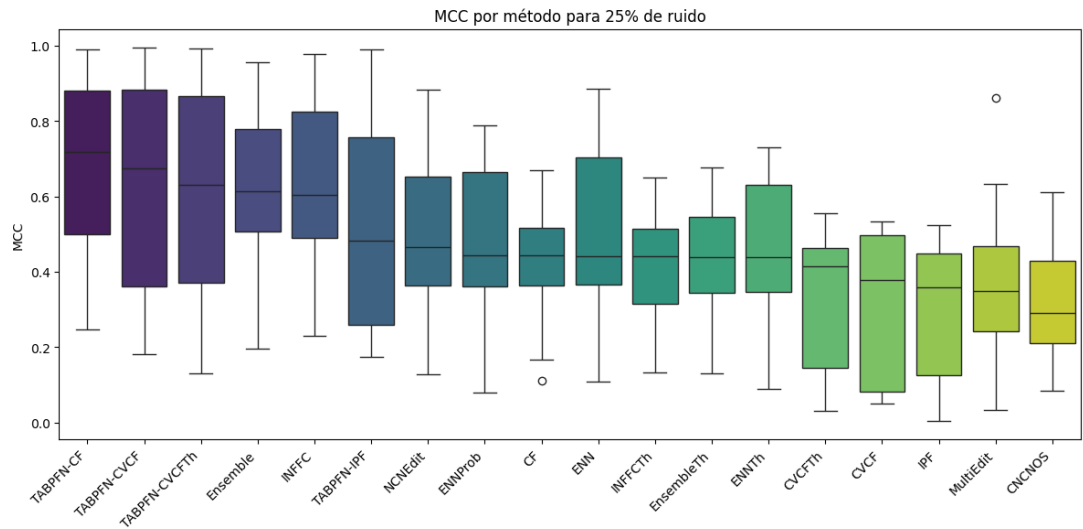


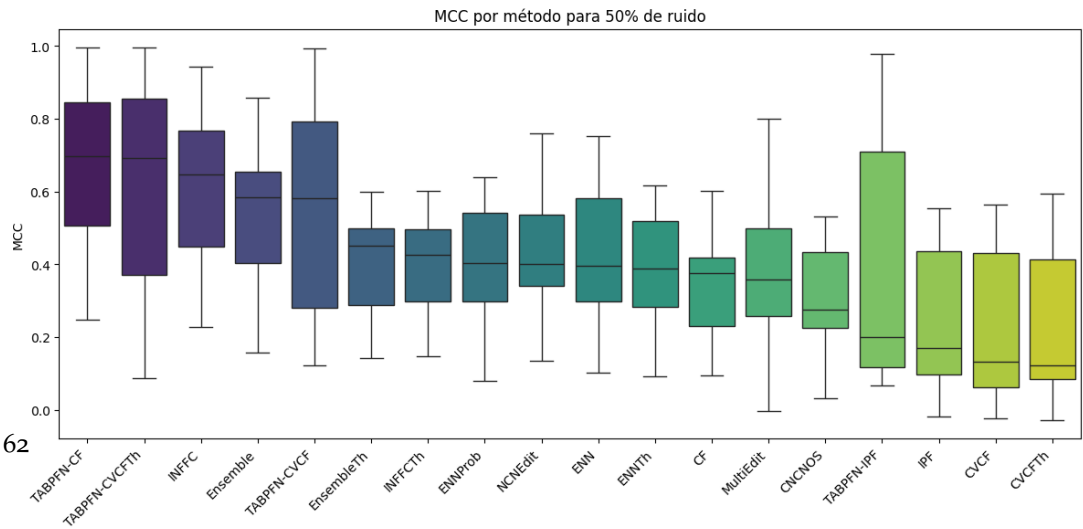
Figure 6.6: Diagramas de cajas sobre el coeficiente de correlación de Matthews por familia y nivel de ruido-



(a)

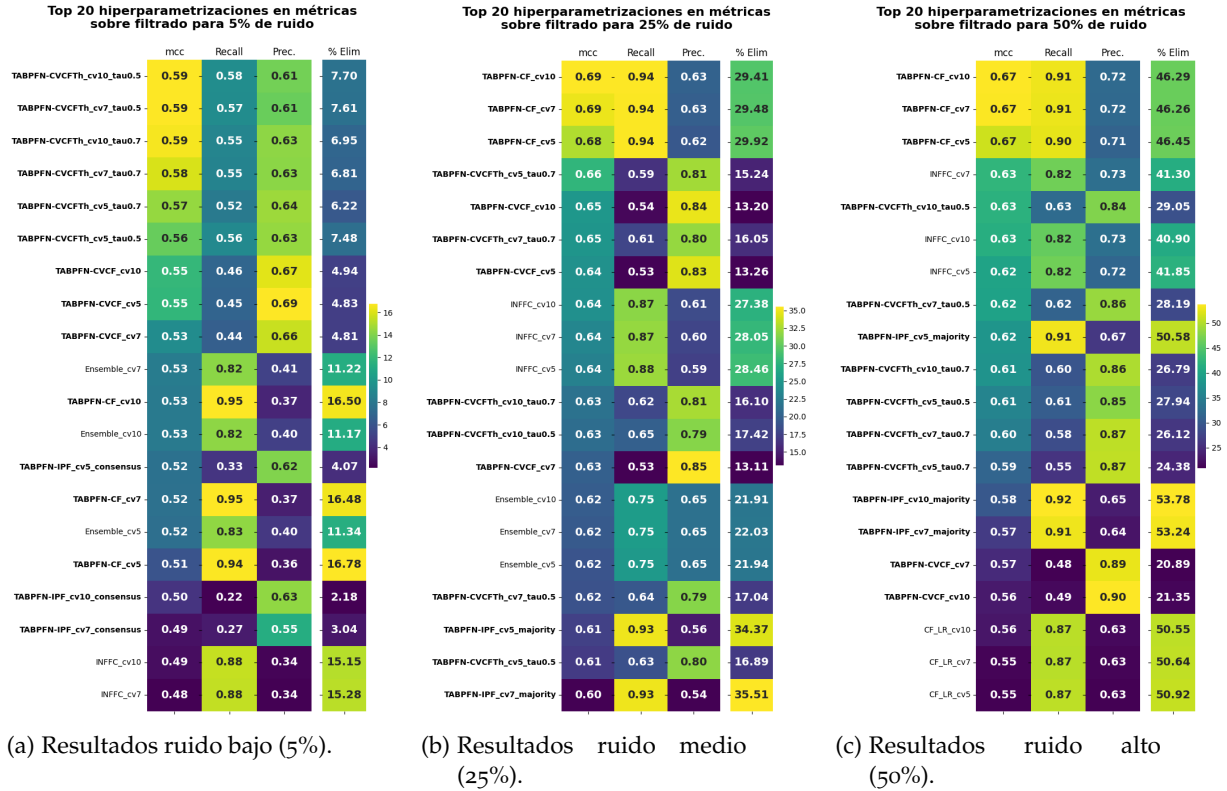


(b)



(c)

Figure 6.7: Desempeño promedio en marcado de instancias ruidosas según la hiperparametrización del filtro y el nivel de ruido inducido.



quis nisl molestie, sed pharetra tellus tempus. Integer facilisis fringilla nisi id efficitur. Nul-
lam nec tincidunt nisi, a lobortis justo. Nunc vitae erat urna. Sed urna dui, tincidunt a velit
sit amet, facilisis placerat magna. Curabitur viverra augue et cursus vehicula.

Sed ornare imperdiet est at eleifend. In eu sagittis diam. Etiam feugiat turpis quis est ele-
mentum, id luctus ligula ornare. Etiam sapien odio, commodo eget tempor quis, maximus
et purus. Donec risus ex, gravida eu faucibus quis, ornare sit amet libero. Sed ut cursus
tortor, eget porta nibh. Integer elementum sit amet diam id eleifend. Duis a ante eu turpis
eleifend ultricies. Pellentesque malesuada in sapien at aliquet. Maecenas viverra scelerisque
elit. Nulla tincidunt nisl nec lacus pellentesque, id interdum risus aliquam. Proin leo dui,
malesuada at justo vitae, auctor pharetra sapien.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam in lacus a nisi tincidunt
congue. Vivamus sed sapien vitae augue aliquam auctor eget vel turpis. Aliquam erat vo-
luptat. Aliquam turpis sapien, dignissim quis nisi eu, sollicitudin ultrices nibh. Vivamus at
feugiat dolor, ut rhoncus sapien. Sed ut voluptat dolor. Vestibulum quis nulla imperdiet,
semper libero ut, sodales dolor. Suspendisse sollicitudin augue vitae leo ultricies, quis condi-
mentum justo elementum. Cras sed sem suscipit nisi voluptat fringilla. Nam eleifend nibh
quis nisl molestie, sed pharetra tellus tempus. Integer facilisis fringilla nisi id efficitur. Nul-

lam nec tincidunt nisi, a lobortis justo. Nunc vitae erat urna. Sed urna dui, tincidunt a velit sit amet, facilisis placerat magna. Curabitur viverra augue et cursus vehicula. Sed ornare imperdiet est at eleifend. In eu sagittis diam. Etiam feugiat turpis quis est elementum, id luctus ligula ornare. Etiam sapien odio, commodo eget tempor quis, maximus et purus. Donec risus ex, gravida eu faucibus quis, ornare sit amet libero. Sed ut cursus tortor, eget porta nibh. Integer elementum sit amet diam id eleifend. Duis a ante eu turpis eleifend ultricies. Pellentesque malesuada in sapien at aliquet. Maecenas viverra scelerisque elit. Nulla tincidunt nisl nec lacus pellentesque, id interdum risus aliquam. Proin leo dui, malesuada at justo vitae, auctor pharetra sapien.

Table 6.2: Top 20 filtros por nivel de ruido

(a) Ruido 5%			(b) Ruido 25%		
Filtro	ζ_{BalAcc}	p_value	Filtro	ζ_{BalAcc}	p_value
TABPFN-CVCFTh_cv5_tauo.5	1.71	1.68E-02	TABPFN-CVCFTh_cv10_tauo.7	4.42	7.51E-04
TABPFN-CVCFTh_cv10_tauo.5	1.68	1.25E-02	TABPFN-CF_cv10	3.92	7.61E-03
TABPFN-CVCFTh_cv7_tauo.5	1.66	1.16E-02	TABPFN-CVCF_cv7	3.91	1.24E-02
TABPFN-CVCFTh_cv10_tauo.7	1.60	6.62E-03	TABPFN-CVCFTh_cv7_tauo.5	3.90	1.08E-02
TABPFN-CVCFTh_cv7_tauo.7	1.60	5.85E-03	TABPFN-CVCFTh_cv5_tauo.7	3.88	2.25E-02
TABPFN-CVCF_cv7	1.41	5.57E-03	TABPFN-CVCF_cv10	3.84	1.88E-02
TABPFN-CVCFTh_cv5_tauo.7	1.39	7.36E-03	TABPFN-CVCFTh_cv10_tauo.5	3.83	2.01E-02
TABPFN-CVCF_cv10	1.37	4.67E-03	TABPFN-CVCFTh_cv5_tauo.5	3.73	1.79E-02
TABPFN-CF_cv5	1.32	6.36E-02	TABPFN-CVCFTh_cv7_tauo.7	3.70	6.17E-03
TABPFN-CF_cv7	1.21	4.73E-02	TABPFN-CF_cv7	3.54	2.21E-02
TABPFN-CF_cv10	1.12	4.73E-02	TABPFN-CVCF_cv5	3.49	1.24E-02
TABPFN-CVCF_cv5	1.10	4.25E-02	TABPFN-CF_cv5	3.29	6.99E-02
Ensemble_cv10	1.08	4.49E-02	Ensemble_cv7	3.24	8.71E-03
Ensemble_cv5	0.95	7.32E-02	Ensemble_cv5	2.90	2.38E-02
CF_DT_cv5	0.85	3.28E-01	Ensemble_cv10	2.79	2.52E-02
CF_c45_cv5	0.85	3.28E-01	EnsembleTh_cv7_tauo.7	2.60	8.41E-02
EnsembleTh_cv10_tauo.7	0.80	9.38E-02	EnsembleTh_cv10_tauo.7	2.49	2.70E-02
TABPFN-IPF_cv5_consensus	0.78	7.30E-02	INFFC_cv10	2.41	5.77E-02
EnsembleTh_cv5_tauo.7	0.72	1.40E-01	EnsembleTh_cv5_tauo.7	2.32	5.23E-02
Ensemble_cv7	0.70	1.46E-01	CF_c45_cv7	2.29	2.45E-01

(c) Ruido 50%

Filtro	ζ_{BalAcc}	p_value
INFFC_cv10	4.28	1.45E-02
INFFC_cv7	3.97	1.29E-01
TABPFN-CVCFTh_cv10_tauo.7	3.79	3.27E-01
TABPFN-CF_cv5	3.76	1.35E-01
INFFC_cv5	3.67	1.05E-01
TABPFN-CF_cv7	3.63	1.46E-01
INFFCTh_cv10_tauo.7	3.49	4.49E-02
TABPFN-CVCFTh_cv7_tauo.5	3.48	4.12E-01
TABPFN-CVCFTh_cv7_tauo.7	3.43	2.55E-01
TABPFN-CVCFTh_cv10_tauo.5	3.43	4.28E-01
TABPFN-CF_cv10	3.42	1.98E-01
TABPFN-CVCF_cv7	3.40	1.28E-01
TABPFN-CVCF_cv10	3.40	1.92E-01
NCNEdit_k9	3.39	1.14E-01
TABPFN-CVCFTh_cv5_tauo.5	3.39	1.51E-01
TABPFN-CVCFTh_cv5_tauo.7	3.36	1.51E-01
ENNProb_k9	3.17	4.17E-01
INFFCTh_cv7_tauo.7	3.12	2.29E-01
Ensemble_cv10	2.95	3.45E-02
EnsembleTh_cv10_tauo.7	2.91	4.05E-02

Table 6.3: Comparación TabPFN-CVCF frente a todos los demás modelos de modo individual.

Method	Mean			p-value		
	5%	25%	50%	5%	25%	50%
<i>LogReg</i>						
CF	-0.49	0.22	-0.55	2.99E-1	6.89E-1	4.65E-1
CNCNOS	4.39	4.93	2.29	1.27E-2	2.47E-2	2.10E-1
CVCF	1.94	4.54	4.39	4.68E-3	3.59E-4	5.77E-2
CVCFTh	2.10	4.73	4.78	1.73E-3	3.59E-4	7.66E-3
ENN	3.18	3.11	1.39	2.34E-2	3.45E-2	3.58E-1
ENNProb	2.40	2.17	0.63	1.58E-1	3.84E-2	6.42E-1
ENNTh	3.59	3.89	1.88	3.09E-1	7.32E-2	1.84E-1
Ensemble	0.48	0.89	0.56	3.71E-1	3.49E-1	9.68E-1
EnsembleTh	0.18	0.79	-0.07	8.08E-1	3.76E-1	9.35E-1
INFFC	1.56	1.62	-0.57	1.91E-1	6.77E-2	7.46E-1
INFFCTh	1.21	2.02	-0.72	2.71E-1	1.39E-1	7.15E-1
IPF	0.81	2.27	0.94	2.31E-1	8.35E-2	7.92E-1
MultiEdit	16.13	15.91	15.03	8.33E-7	5.22E-8	3.37E-6
NCNEdit	2.74	3.12	0.25	4.05E-2	3.50E-2	5.56E-1
TABPFN-CF	0.15	0.37	0.37	4.66E-1	8.73E-1	4.69E-1
TABPFN-CVCF	0.47	0.65	0.84	2.32E-1	2.40E-2	5.77E-2
TABPFN-IPF	0.21	1.97	1.16	7.32E-1	3.02E-1	5.69E-1

7 Conclusiones

7.1 Conclusiones generales

7.2 Cumplimiento de objetivos

7.3 Limitaciones del trabajo

7.4 Líneas futuras

Bibliography

- [1] Jesús Alcalá-Fdez, Luciano Sánchez, Salvador García, María José del Jesus, Sebastián Ventura, Josep Maria Garrell, José Otero, Cristóbal Romero, Jaume Bacardit, Víctor Manuel Rivas, Juan Carlos Fernández, and Francisco Herrera. KEEL: A software tool to assess evolutionary algorithms to data mining problems. *Soft Computing*, 13(3):307–318, 2009.
- [2] Alaa Althubaiti. Information bias in health research: definition, pitfalls, and adjustment methods. 9:211–217.
- [3] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- [4] Carla E. Brodley and Mark A. Friedl. Identifying mislabeled training data. *Journal of Artificial Intelligence Research*, 11:131–167, 1999.
- [5] Cleanlab. cleanlab: The standard data-centric ai package for data quality and machine learning with noisy labels. <https://docs.cleanlab.ai/stable/index.html>, 2026. Python package documentation.
- [6] Cleanlab. CleanVision: Find issues in image datasets. <https://pypi.org/project/cleanvision/>, 2026. Python package, version 0.3.7.
- [7] William W. Cohen. Fast effective rule induction. In *Proceedings of the Twelfth International Conference on Machine Learning*, pages 115–123. Morgan Kaufmann, 1995.
- [8] Benoît Frénay and Michel Verleysen. Classification in the presence of label noise: A survey. *IEEE Transactions on Neural Networks and Learning Systems*, 25(5):845–869, 2014.
- [9] Dragan Gamberger, Nada Lavrač, and Ciril Grošelj. Experiments with noise filtering in a medical domain. In *Proceedings of the Sixteenth International Conference on Machine Learning*, pages 143–151. Morgan Kaufmann, 1999.
- [10] Haibo He and Edwardo A. Garcia. Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9):1263–1284, 2009.
- [11] Noah Hollmann, Samuel Müller, Katharina Eggenberger, and Frank Hutter. Tabpfn: A transformer that solves small tabular classification problems in a second, 2023.
- [12] Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani. *An Introduction to Statistical Learning: With Applications in R*. Springer, 2 edition, 2021.
- [13] Sotiris B. Kotsiantis, Dimitris Kanellopoulos, and Panayiotis E. Pintelas. Data preprocessing for supervised learning. *International Journal of Computer Science*, 1(2):111–117, 2006.
- [14] Guillaume Lemaître, Fernando Nogueira, and Christos K. Aridas. Imbalanced-learn: A python toolbox to tackle the curse of imbalanced datasets in machine learning. *Journal of Machine Learning Research*, 18(17):1–5, 2017.

- [15] Zachary C. Lipton. The mythos of model interpretability. *Queue*, 16(3):31–57, 2018.
- [16] Julián Luengo, Seong-O Shim, Saleh Alshomrani, Abdulrahman Altalhi, and Francisco Herrera. Cnc-nos: Class noise cleaning by ensemble filtering and noise scoring. *Knowledge-Based Systems*, 140:27–49, 2018.
- [17] Scott Lundberg and Su-In Lee. A unified approach to interpreting model predictions, 2017.
- [18] Ninareh Mehrabi, Fred Morstatter, Nripsuta Saxena, Kristina Lerman, and Aram Galstyan. A survey on bias and fairness in machine learning. *ACM Computing Surveys*, 54(6):1–35, 2021.
- [19] Tom M. Mitchell. *Machine Learning*. McGraw-Hill, 1997.
- [20] Pablo Morales, Julián Luengo, Luís P. F. Garcia, Ana C. Lorena, André C. P. L. F. de Carvalho, and Francisco Herrera. The NoiseFiltersR package: Label noise preprocessing in R. *The R Journal*, 9(1):219–228, 2017.
- [21] Curtis G. Northcutt, Lu Jiang, and Isaac L. Chuang. Confident learning: Estimating uncertainty in dataset labels. *Journal of Artificial Intelligence Research*, 70:1373–1411, 2021.
- [22] J. Ross Quinlan. *C4.5: Programs for Machine Learning*. Morgan Kaufmann, 1993.
- [23] Nithya Sambasivan, Shivani Kapania, Hannah Highfill, Diana Akrong, Praveen Kumar Paritosh, and Lora Mois Aroyo. "everyone wants to do the model work, not the data work": Data cascades in high-stakes ai. 2021.
- [24] Seagate Technology. Rethink data: Put more of your business data to work – from edge to cloud. Technical report, Seagate Technology and IDC, 2020.
- [25] Shihab Shahriar. scikit-clean: A python machine learning library for classification in the presence of label noise. <https://pypi.org/project/scikit-clean/>, 2020. Python package, version 0.1.2.
- [26] Diane M. Strong, Yang W. Lee, and Richard Y. Wang. Data quality in context. *Communications of the ACM*, 40(5):103–110, 1997.
- [27] Harini Suresh and John V. Guttag. A framework for understanding sources of harm throughout the machine learning life cycle. In *Proceedings of the 1st ACM Conference on Equity and Access in Algorithms, Mechanisms, and Optimization*, pages 1–9, 2021.
- [28] Sofie Verbaeten and Anneleen Van Assche. Ensemble methods for noise elimination in classification problems. In *International Workshop on Multiple Classifier Systems*, pages 317–325. Springer, 2003.
- [29] Yair Wand and Richard Y. Wang. Anchoring data quality dimensions in ontological foundations. *Communications of the ACM*, 39(11):86–95, 1996.
- [30] Richard Y. Wang and Diane M. Strong. Beyond accuracy: What data quality means to data consumers. *Journal of Management Information Systems*, 12(4):5–33, 1996.
- [31] Xindong Wu, Vipin Kumar, J. Ross Quinlan, Joydeep Ghosh, Qiang Yang, Hiroshi Motoda, Geoffrey J. McLachlan, Angus Ng, Bing Liu, Philip S. Yu, et al. Top 10 algorithms in data mining. *Knowledge and Information Systems*, 14(1):1–37, 2008.

- [32] Xingquan Zhu and Xindong Wu. Class noise vs. attribute noise: A quantitative study. *Artificial Intelligence Review*, 22:177–210, 2004.